

pruned by cross-validation 8.7%, and MARS 5.5%. The standard error of these estimates is around 0.6%, although gradient boosting is significantly better than all of them using the McNemar test (Exercise 10.6).

In Section 10.13 below we develop a relative importance measure for each predictor, as well as a partial dependence plot describing a predictor's contribution to the fitted model. We now illustrate these for the spam data.

Figure 10.6 displays the relative importance spectrum for all 57 predictor variables. Clearly some predictors are more important than others in separating **spam** from **email**. The frequencies of the character strings **!**, **\$**, **hp**, and **remove** are estimated to be the four most relevant predictor variables. At the other end of the spectrum, the character strings **857**, **415**, **table**, and **3d** have virtually no relevance.

The quantity being modeled here is the log-odds of **spam** versus **email**

$$f(x) = \log \frac{\Pr(\text{spam}|x)}{\Pr(\text{email}|x)} \quad (10.24)$$

(see Section 10.13 below). Figure 10.7 shows the partial dependence of the log-odds on selected important predictors, two positively associated with **spam** (**!** and **remove**), and two negatively associated (**edu** and **hp**). These particular dependencies are seen to be essentially monotonic. There is a general agreement with the corresponding functions found by the additive logistic regression model; see Figure 9.1 on page 303.

Running a gradient boosted model on these data with $J = 2$ terminal-node trees produces a purely additive (main effects) model for the log-odds, with a corresponding error rate of 4.7%, as compared to 4.5% for the full gradient boosted model (with $J = 5$ terminal-node trees). Although not significant, this slightly higher error rate suggests that there may be interactions among some of the important predictor variables. This can be diagnosed through two-variable partial dependence plots. Figure 10.8 shows one of the several such plots displaying strong interaction effects.

One sees that for very low frequencies of **hp**, the log-odds of **spam** are greatly increased. For high frequencies of **hp**, the log-odds of **spam** tend to be much lower and roughly constant as a function of **!**. As the frequency of **hp** decreases, the functional relationship with **!** strengthens.

10.9 Boosting Trees

Regression and classification trees are discussed in detail in Section 9.2. They partition the space of all joint predictor variable values into disjoint regions R_j , $j = 1, 2, \dots, J$, as represented by the terminal nodes of the tree. A constant γ_j is assigned to each such region and the predictive rule is

$$x \in R_j \Rightarrow f(x) = \gamma_j.$$

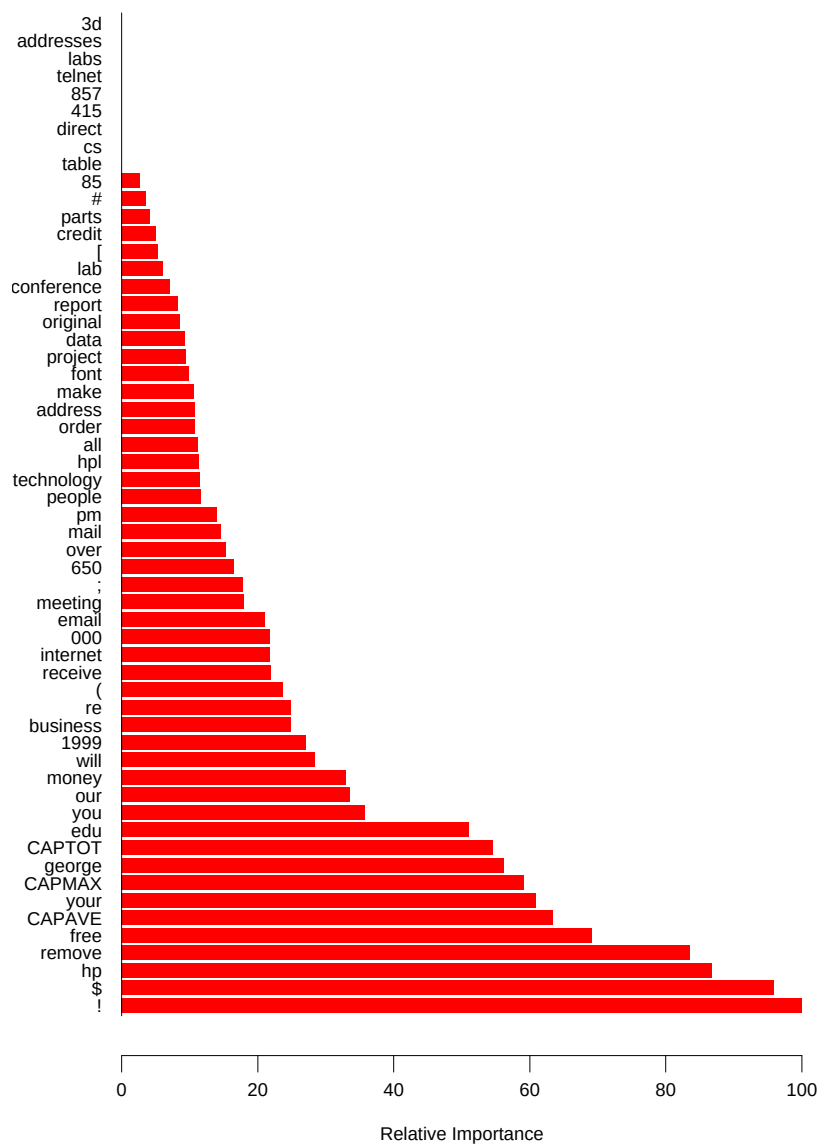


FIGURE 10.6. Predictor variable importance spectrum for the `spam` data. The variable names are written on the vertical axis.

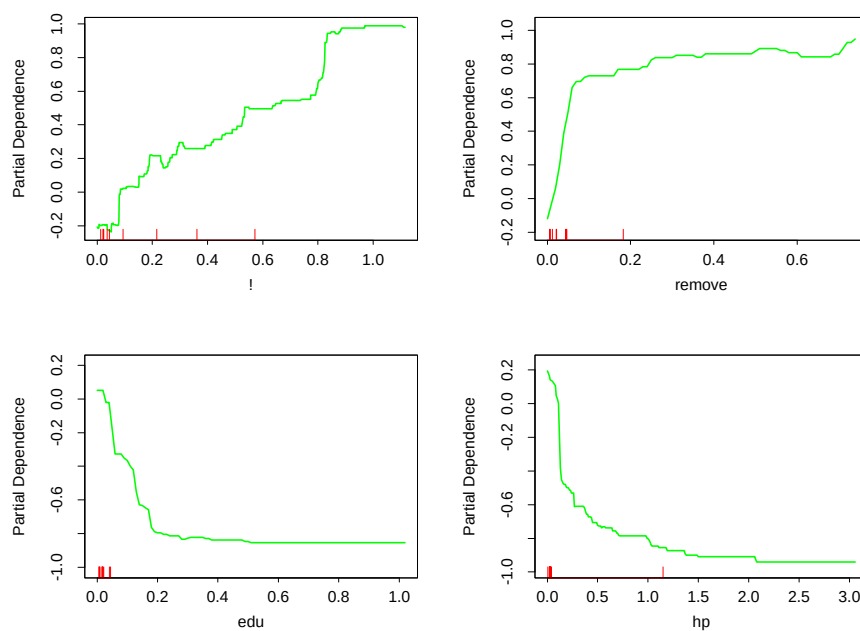


FIGURE 10.7. Partial dependence of log-odds of `spam` on four important predictors. The red ticks at the base of the plots are deciles of the input variable.

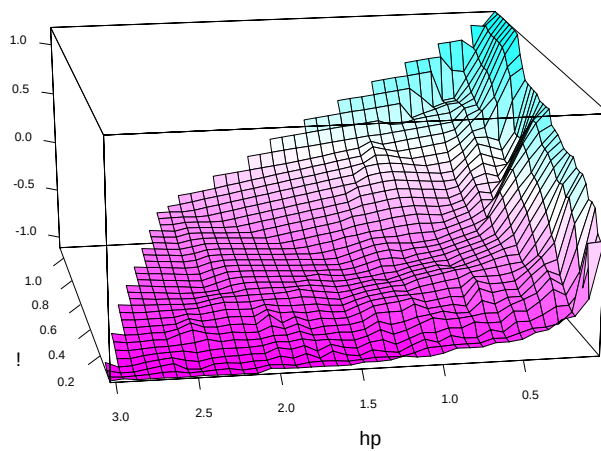


FIGURE 10.8. Partial dependence of the log-odds of `spam` vs. `email` as a function of joint frequencies of `hp` and the character `!`.

Thus a tree can be formally expressed as

$$T(x; \Theta) = \sum_{j=1}^J \gamma_j I(x \in R_j), \quad (10.25)$$

with parameters $\Theta = \{R_j, \gamma_j\}_1^J$. J is usually treated as a meta-parameter. The parameters are found by minimizing the empirical risk

$$\hat{\Theta} = \arg \min_{\Theta} \sum_{j=1}^J \sum_{x_i \in R_j} L(y_i, \gamma_j). \quad (10.26)$$

This is a formidable combinatorial optimization problem, and we usually settle for approximate suboptimal solutions. It is useful to divide the optimization problem into two parts:

Finding γ_j given R_j : Given the R_j , estimating the γ_j is typically trivial, and often $\hat{\gamma}_j = \bar{y}_j$, the mean of the y_i falling in region R_j . For misclassification loss, $\hat{\gamma}_j$ is the modal class of the observations falling in region R_j .

Finding R_j : This is the difficult part, for which approximate solutions are found. Note also that finding the R_j entails estimating the γ_j as well. A typical strategy is to use a greedy, top-down recursive partitioning algorithm to find the R_j . In addition, it is sometimes necessary to approximate (10.26) by a smoother and more convenient criterion for optimizing the R_j :

$$\tilde{\Theta} = \arg \min_{\Theta} \sum_{i=1}^N \tilde{L}(y_i, T(x_i, \Theta)). \quad (10.27)$$

Then given the $\hat{R}_j = \tilde{R}_j$, the γ_j can be estimated more precisely using the original criterion.

In Section 9.2 we described such a strategy for classification trees. The Gini index replaced misclassification loss in the growing of the tree (identifying the R_j).

The boosted tree model is a sum of such trees,

$$f_M(x) = \sum_{m=1}^M T(x; \Theta_m), \quad (10.28)$$

induced in a forward stagewise manner (Algorithm 10.2). At each step in the forward stagewise procedure one must solve

$$\hat{\Theta}_m = \arg \min_{\Theta_m} \sum_{i=1}^N L(y_i, f_{m-1}(x_i) + T(x_i; \Theta_m)) \quad (10.29)$$

for the region set and constants $\Theta_m = \{R_{jm}, \gamma_{jm}\}_1^{J_m}$ of the next tree, given the current model $f_{m-1}(x)$.

Given the regions R_{jm} , finding the optimal constants γ_{jm} in each region is typically straightforward:

$$\hat{\gamma}_{jm} = \arg \min_{\gamma_{jm}} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma_{jm}). \quad (10.30)$$

Finding the regions is difficult, and even more difficult than for a single tree. For a few special cases, the problem simplifies.

For squared-error loss, the solution to (10.29) is no harder than for a single tree. It is simply the regression tree that best predicts the current residuals $y_i - f_{m-1}(x_i)$, and $\hat{\gamma}_{jm}$ is the mean of these residuals in each corresponding region.

For two-class classification and exponential loss, this stagewise approach gives rise to the AdaBoost method for boosting classification trees (Algorithm 10.1). In particular, if the trees $T(x; \Theta_m)$ are restricted to be scaled classification trees, then we showed in Section 10.4 that the solution to (10.29) is the tree that minimizes the weighted error rate $\sum_{i=1}^N w_i^{(m)} I(y_i \neq T(x_i; \Theta_m))$ with weights $w_i^{(m)} = e^{-y_i f_{m-1}(x_i)}$. By a scaled classification tree, we mean $\beta_m T(x; \Theta_m)$, with the restriction that $\gamma_{jm} \in \{-1, 1\}$.

Without this restriction, (10.29) still simplifies for exponential loss to a weighted exponential criterion for the new tree:

$$\hat{\Theta}_m = \arg \min_{\Theta_m} \sum_{i=1}^N w_i^{(m)} \exp[-y_i T(x_i; \Theta_m)]. \quad (10.31)$$

It is straightforward to implement a greedy recursive-partitioning algorithm using this weighted exponential loss as a splitting criterion. Given the R_{jm} , one can show (Exercise 10.7) that the solution to (10.30) is the weighted log-odds in each corresponding region

$$\hat{\gamma}_{jm} = \frac{1}{2} \log \frac{\sum_{x_i \in R_{jm}} w_i^{(m)} I(y_i = 1)}{\sum_{x_i \in R_{jm}} w_i^{(m)} I(y_i = -1)}. \quad (10.32)$$

This requires a specialized tree-growing algorithm; in practice, we prefer the approximation presented below that uses a weighted least squares regression tree.

Using loss criteria such as the absolute error or the Huber loss (10.23) in place of squared-error loss for regression, and the deviance (10.22) in place of exponential loss for classification, will serve to robustify boosting trees. Unfortunately, unlike their nonrobust counterparts, these robust criteria do not give rise to simple fast boosting algorithms.

For more general loss criteria the solution to (10.30), given the R_{jm} , is typically straightforward since it is a simple “location” estimate. For

absolute loss it is just the median of the residuals in each respective region. For the other criteria fast iterative algorithms exist for solving (10.30), and usually their faster “single-step” approximations are adequate. The problem is tree induction. Simple fast algorithms do not exist for solving (10.29) for these more general loss criteria, and approximations like (10.27) become essential.

10.10 Numerical Optimization via Gradient Boosting

Fast approximate algorithms for solving (10.29) with any differentiable loss criterion can be derived by analogy to numerical optimization. The loss in using $f(x)$ to predict y on the training data is

$$L(f) = \sum_{i=1}^N L(y_i, f(x_i)). \quad (10.33)$$

The goal is to minimize $L(f)$ with respect to f , where here $f(x)$ is constrained to be a sum of trees (10.28). Ignoring this constraint, minimizing (10.33) can be viewed as a numerical optimization

$$\hat{\mathbf{f}} = \arg \min_{\mathbf{f}} L(\mathbf{f}), \quad (10.34)$$

where the “parameters” $\mathbf{f} \in \mathbb{R}^N$ are the values of the approximating function $f(x_i)$ at each of the N data points x_i :

$$\mathbf{f} = \{f(x_1), f(x_2), \dots, f(x_N)\}^T.$$

Numerical optimization procedures solve (10.34) as a sum of component vectors

$$\mathbf{f}_M = \sum_{m=0}^M \mathbf{h}_m, \quad \mathbf{h}_m \in \mathbb{R}^N,$$

where $\mathbf{f}_0 = \mathbf{h}_0$ is an initial guess, and each successive $\mathbf{f}_m$ is induced based on the current parameter vector $\mathbf{f}_{m-1}$, which is the sum of the previously induced updates. Numerical optimization methods differ in their prescriptions for computing each increment vector $\mathbf{h}_m$ (“step”).

10.10.1 Steepest Descent

Steepest descent chooses $\mathbf{h}_m = -\rho_m \mathbf{g}_m$ where ρ_m is a scalar and $\mathbf{g}_m \in \mathbb{R}^N$ is the gradient of $L(\mathbf{f})$ evaluated at $\mathbf{f} = \mathbf{f}_{m-1}$. The components of the gradient $\mathbf{g}_m$ are

$$g_{im} = \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f(x_i)=f_{m-1}(x_i)} \quad (10.35)$$

The *step length* ρ_m is the solution to

$$\rho_m = \arg \min_{\rho} L(\mathbf{f}_{m-1} - \rho \mathbf{g}_m). \quad (10.36)$$

The current solution is then updated

$$\mathbf{f}_m = \mathbf{f}_{m-1} - \rho_m \mathbf{g}_m$$

and the process repeated at the next iteration. Steepest descent can be viewed as a very greedy strategy, since $-\mathbf{g}_m$ is the local direction in $\mathbb{R}^N$ for which $L(\mathbf{f})$ is most rapidly decreasing at $\mathbf{f} = \mathbf{f}_{m-1}$.

10.10.2 Gradient Boosting

Forward stagewise boosting (Algorithm 10.2) is also a very greedy strategy. At each step the solution tree is the one that maximally reduces (10.29), given the current model f_{m-1} and its fits $f_{m-1}(x_i)$. Thus, the tree predictions $T(x_i; \Theta_m)$ are analogous to the components of the negative gradient (10.35). The principal difference between them is that the tree components $\mathbf{t}_m = \{T(x_1; \Theta_m), \dots, T(x_N; \Theta_m)\}^T$ are not independent. They are constrained to be the predictions of a J_m -terminal node decision tree, whereas the negative gradient is the unconstrained maximal descent direction.

The solution to (10.30) in the stagewise approach is analogous to the line search (10.36) in steepest descent. The difference is that (10.30) performs a separate line search for those components of $\mathbf{t}_m$ that correspond to each separate terminal region $\{T(x_i; \Theta_m)\}_{x_i \in R_{jm}}$.

If minimizing loss on the training data (10.33) were the only goal, steepest descent would be the preferred strategy. The gradient (10.35) is trivial to calculate for any differentiable loss function $L(y, f(x))$, whereas solving (10.29) is difficult for the robust criteria discussed in Section 10.6. Unfortunately the gradient (10.35) is defined only at the training data points x_i , whereas the ultimate goal is to generalize $f_M(x)$ to new data not represented in the training set.

A possible resolution to this dilemma is to induce a tree $T(x; \Theta_m)$ at the m th iteration whose predictions $\mathbf{t}_m$ are as close as possible to the negative gradient. Using squared error to measure closeness, this leads us to

$$\tilde{\Theta}_m = \arg \min_{\Theta} \sum_{i=1}^N (-g_{im} - T(x_i; \Theta))^2. \quad (10.37)$$

That is, one fits the tree T to the negative gradient values (10.35) by least squares. As noted in Section 10.9 fast algorithms exist for least squares decision tree induction. Although the solution regions $\tilde{R}_{jm}$ to (10.37) will not be identical to the regions R_{jm} that solve (10.29), it is generally similar enough to serve the same purpose. In any case, the forward stagewise

TABLE 10.2. *Gradients for commonly used loss functions.*

Setting	Loss Function	$-\partial L(y_i, f(x_i))/\partial f(x_i)$
Regression	$\frac{1}{2}[y_i - f(x_i)]^2$	$y_i - f(x_i)$
Regression	$ y_i - f(x_i) $	$\text{sign}[y_i - f(x_i)]$
Regression	Huber	$y_i - f(x_i)$ for $ y_i - f(x_i) \leq \delta_m$ $\delta_m \text{sign}[y_i - f(x_i)]$ for $ y_i - f(x_i) > \delta_m$ where $\delta_m = \alpha\text{th-quantile}\{ y_i - f(x_i) \}$
Classification	Deviance	$k\text{th component: } I(y_i = \mathcal{G}_k) - p_k(x_i)$

boosting procedure, and top-down decision tree induction, are themselves approximation procedures. After constructing the tree (10.37), the corresponding constants in each region are given by (10.30).

Table 10.2 summarizes the gradients for commonly used loss functions. For squared error loss, the negative gradient is just the ordinary residual $-g_{im} = y_i - f_{m-1}(x_i)$, so that (10.37) on its own is equivalent to standard least-squares boosting. With absolute error loss, the negative gradient is the *sign* of the residual, so at each iteration (10.37) fits the tree to the sign of the current residuals by least squares. For Huber M-regression, the negative gradient is a compromise between these two (see the table).

For classification the loss function is the multinomial deviance (10.22), and K least squares trees are constructed at each iteration. Each tree T_{km} is fit to its respective negative gradient vector $\mathbf{g}_{km}$,

$$\begin{aligned}
 -g_{ikm} &= \left[\frac{\partial L(y_i, f_1(x_i), \dots, f_K(x_i))}{\partial f_k(x_i)} \right]_{\mathbf{f}(x_i) = \mathbf{f}_{m-1}(x_i)} \\
 &= I(y_i = \mathcal{G}_k) - p_k(x_i),
 \end{aligned} \tag{10.38}$$

with $p_k(x)$ given by (10.21). Although K separate trees are built at each iteration, they are related through (10.21). For binary classification ($K = 2$), only one tree is needed (exercise 10.10).

10.10.3 Implementations of Gradient Boosting

Algorithm 10.3 presents the generic gradient tree-boosting algorithm for regression. Specific algorithms are obtained by inserting different loss criteria $L(y, f(x))$. The first line of the algorithm initializes to the optimal constant model, which is just a single terminal node tree. The components of the negative gradient computed at line 2(a) are referred to as generalized or *pseudo* residuals, r . Gradients for commonly used loss functions are summarized in Table 10.2.

Algorithm 10.3 *Gradient Tree Boosting Algorithm.*

1. Initialize $f_0(x) = \arg \min_{\gamma} \sum_{i=1}^N L(y_i, \gamma)$.

2. For $m = 1$ to M :

(a) For $i = 1, 2, \dots, N$ compute

$$r_{im} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f=f_{m-1}}.$$

(b) Fit a regression tree to the targets r_{im} giving terminal regions R_{jm} , $j = 1, 2, \dots, J_m$.

(c) For $j = 1, 2, \dots, J_m$ compute

$$\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma).$$

(d) Update $f_m(x) = f_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$.

3. Output $\hat{f}(x) = f_M(x)$.

The algorithm for classification is similar. Lines 2(a)–(d) are repeated K times at each iteration m , once for each class using (10.38). The result at line 3 is K different (coupled) tree expansions $f_{kM}(x)$, $k = 1, 2, \dots, K$. These produce probabilities via (10.21) or do classification as in (10.20). Details are given in Exercise 10.9. Two basic tuning parameters are the number of iterations M and the sizes of each of the constituent trees J_m , $m = 1, 2, \dots, M$.

The original implementation of this algorithm was called MART for “multiple additive regression trees,” and was referred to in the first edition of this book. Many of the figures in this chapter were produced by MART. Gradient boosting as described here is implemented in the R `gbm` package (Ridgeway, 1999, “Gradient Boosted Models”), and is freely available. The `gbm` package is used in Section 10.14.2, and extensively in Chapters 16 and 15. Another R implementation of boosting is `mboost` (Hothorn and Bühlmann, 2006). A commercial implementation of gradient boosting/MART called *TreeNet*[®] is available from Salford Systems, Inc.

10.11 Right-Sized Trees for Boosting

Historically, boosting was considered to be a technique for combining models, here trees. As such, the tree building algorithm was regarded as a

primitive that produced models to be combined by the boosting procedure. In this scenario, the optimal size of each tree is estimated separately in the usual manner when it is built (Section 9.2). A very large (oversized) tree is first induced, and then a bottom-up procedure is employed to prune it to the estimated optimal number of terminal nodes. This approach assumes implicitly that each tree is the last one in the expansion (10.28). Except perhaps for the very last tree, this is clearly a very poor assumption. The result is that trees tend to be much too large, especially during the early iterations. This substantially degrades performance and increases computation.

The simplest strategy for avoiding this problem is to restrict all trees to be the same size, $J_m = J \forall m$. At each iteration a J -terminal node regression tree is induced. Thus J becomes a meta-parameter of the entire boosting procedure, to be adjusted to maximize estimated performance for the data at hand.

One can get an idea of useful values for J by considering the properties of the “target” function

$$\eta = \arg \min_f E_{XY} L(Y, f(X)). \quad (10.39)$$

Here the expected value is over the population joint distribution of (X, Y) . The target function $\eta(x)$ is the one with minimum prediction risk on future data. This is the function we are trying to approximate.

One relevant property of $\eta(X)$ is the degree to which the coordinate variables $X^T = (X_1, X_2, \dots, X_p)$ interact with one another. This is captured by its ANOVA (analysis of variance) expansion

$$\eta(X) = \sum_j \eta_j(X_j) + \sum_{jk} \eta_{jk}(X_j, X_k) + \sum_{jkl} \eta_{jkl}(X_j, X_k, X_l) + \dots \quad (10.40)$$

The first sum in (10.40) is over functions of only a single predictor variable X_j . The particular functions $\eta_j(X_j)$ are those that jointly best approximate $\eta(X)$ under the loss criterion being used. Each such $\eta_j(X_j)$ is called the “main effect” of X_j . The second sum is over those two-variable functions that when added to the main effects best fit $\eta(X)$. These are called the second-order interactions of each respective variable pair (X_j, X_k) . The third sum represents third-order interactions, and so on. For many problems encountered in practice, low-order interaction effects tend to dominate. When this is the case, models that produce strong higher-order interaction effects, such as large decision trees, suffer in accuracy.

The interaction level of tree-based approximations is limited by the tree size J . Namely, no interaction effects of level greater than $J - 1$ are possible. Since boosted models are additive in the trees (10.28), this limit extends to them as well. Setting $J = 2$ (single split “decision stump”) produces boosted models with only main effects; no interactions are permitted. With $J = 3$, two-variable interaction effects are also allowed, and

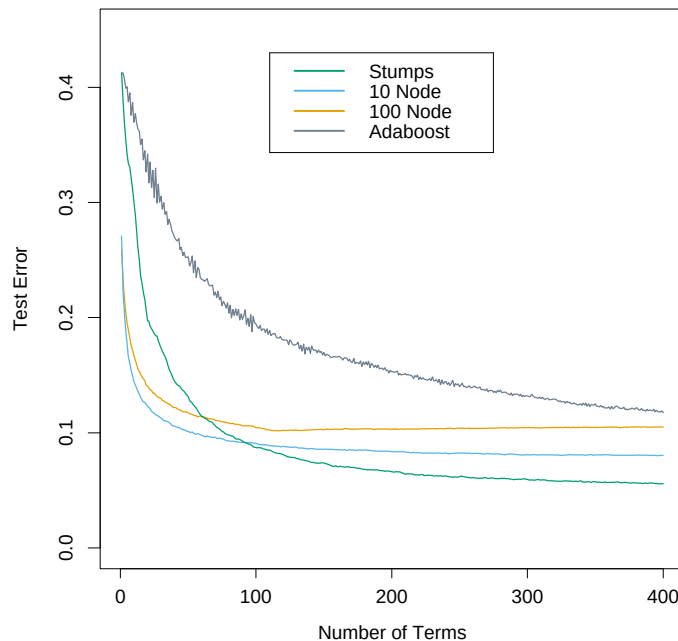


FIGURE 10.9. Boosting with different sized trees, applied to the example (10.2) used in Figure 10.2. Since the generative model is additive, stumps perform the best. The boosting algorithm used the binomial deviance loss in Algorithm 10.3; shown for comparison is the AdaBoost Algorithm 10.1.

so on. This suggests that the value chosen for J should reflect the level of dominant interactions of $\eta(x)$. This is of course generally unknown, but in most situations it will tend to be low. Figure 10.9 illustrates the effect of interaction order (choice of J) on the simulation example (10.2). The generative function is additive (sum of quadratic monomials), so boosting models with $J > 2$ incurs unnecessary variance and hence the higher test error. Figure 10.10 compares the coordinate functions found by boosted stumps with the true functions.

Although in many applications $J = 2$ will be insufficient, it is unlikely that $J > 10$ will be required. Experience so far indicates that $4 \leq J \leq 8$ works well in the context of boosting, with results being fairly insensitive to particular choices in this range. One can fine-tune the value for J by trying several different values and choosing the one that produces the lowest risk on a validation sample. However, this seldom provides significant improvement over using $J \simeq 6$.

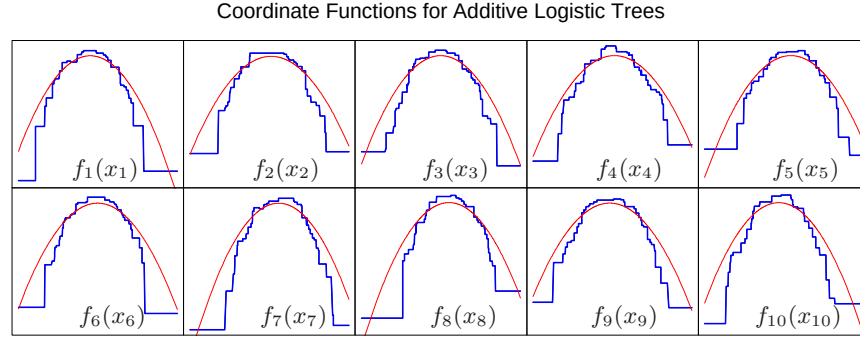


FIGURE 10.10. Coordinate functions estimated by boosting stumps for the simulated example used in Figure 10.9. The true quadratic functions are shown for comparison.

10.12 Regularization

Besides the size of the constituent trees, J , the other meta-parameter of gradient boosting is the number of boosting iterations M . Each iteration usually reduces the training risk $L(f_M)$, so that for M large enough this risk can be made arbitrarily small. However, fitting the training data too well can lead to overfitting, which degrades the risk on future predictions. Thus, there is an optimal number M^* minimizing future risk that is application dependent. A convenient way to estimate M^* is to monitor prediction risk as a function of M on a validation sample. The value of M that minimizes this risk is taken to be an estimate of M^* . This is analogous to the early stopping strategy often used with neural networks (Section 11.4).

10.12.1 Shrinkage

Controlling the value of M is not the only possible regularization strategy. As with ridge regression and neural networks, shrinkage techniques can be employed as well (see Sections 3.4.1 and 11.5). The simplest implementation of shrinkage in the context of boosting is to scale the contribution of each tree by a factor $0 < \nu < 1$ when it is added to the current approximation. That is, line 2(d) of Algorithm 10.3 is replaced by

$$f_m(x) = f_{m-1}(x) + \nu \cdot \sum_{j=1}^J \gamma_{jm} I(x \in R_{jm}). \quad (10.41)$$

The parameter ν can be regarded as controlling the learning rate of the boosting procedure. Smaller values of ν (more shrinkage) result in larger training risk for the same number of iterations M . Thus, both ν and M control prediction risk on the training data. However, these parameters do

not operate independently. Smaller values of ν lead to larger values of M for the same training risk, so that there is a tradeoff between them.

Empirically it has been found (Friedman, 2001) that smaller values of ν favor better test error, and require correspondingly larger values of M . In fact, the best strategy appears to be to set ν to be very small ($\nu < 0.1$) and then choose M by early stopping. This yields dramatic improvements (over no shrinkage $\nu = 1$) for regression and for probability estimation. The corresponding improvements in misclassification risk via (10.20) are less, but still substantial. The price paid for these improvements is computational: smaller values of ν give rise to larger values of M , and computation is proportional to the latter. However, as seen below, many iterations are generally computationally feasible even on very large data sets. This is partly due to the fact that small trees are induced at each step with no pruning.

Figure 10.11 shows test error curves for the simulated example (10.2) of Figure 10.2. A gradient boosted model (MART) was trained using binomial deviance, using either stumps or six terminal-node trees, and with or without shrinkage. The benefits of shrinkage are evident, especially when the binomial deviance is tracked. With shrinkage, each test error curve reaches a lower value, and stays there for many iterations.

Section 16.2.1 draws a connection between forward stagewise shrinkage in boosting and the use of an L_1 penalty for regularizing model parameters (the “lasso”). We argue that L_1 penalties may be superior to the L_2 penalties used by methods such as the support vector machine.

10.12.2 Subsampling

We saw in Section 8.7 that bootstrap averaging (bagging) improves the performance of a noisy classifier through averaging. Chapter 15 discusses in some detail the variance-reduction mechanism of this sampling followed by averaging. We can exploit the same device in gradient boosting, both to improve performance and computational efficiency.

With *stochastic gradient boosting* (Friedman, 1999), at each iteration we sample a fraction η of the training observations (without replacement), and grow the next tree using that subsample. The rest of the algorithm is identical. A typical value for η can be $\frac{1}{2}$, although for large N , η can be substantially smaller than $\frac{1}{2}$.

Not only does the sampling reduce the computing time by the same fraction η , but in many cases it actually produces a more accurate model.

Figure 10.12 illustrates the effect of subsampling using the simulated example (10.2), both as a classification and as a regression example. We see in both cases that sampling along with shrinkage slightly outperformed the rest. It appears here that subsampling without shrinkage does poorly.

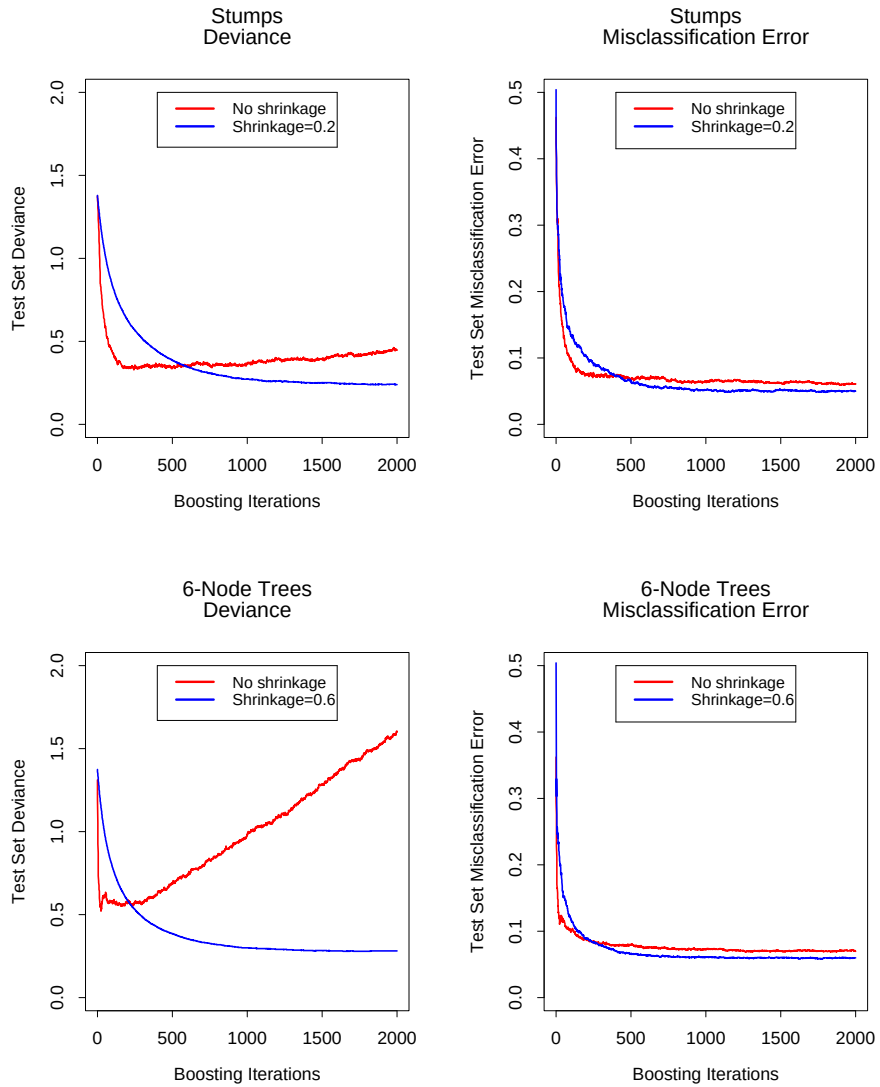


FIGURE 10.11. Test error curves for simulated example (10.2) of Figure 10.9, using gradient boosting (MART). The models were trained using binomial deviance, either stumps or six terminal-node trees, and with or without shrinkage. The left panels report test deviance, while the right panels show misclassification error. The beneficial effect of shrinkage can be seen in all cases, especially for deviance in the left panels.

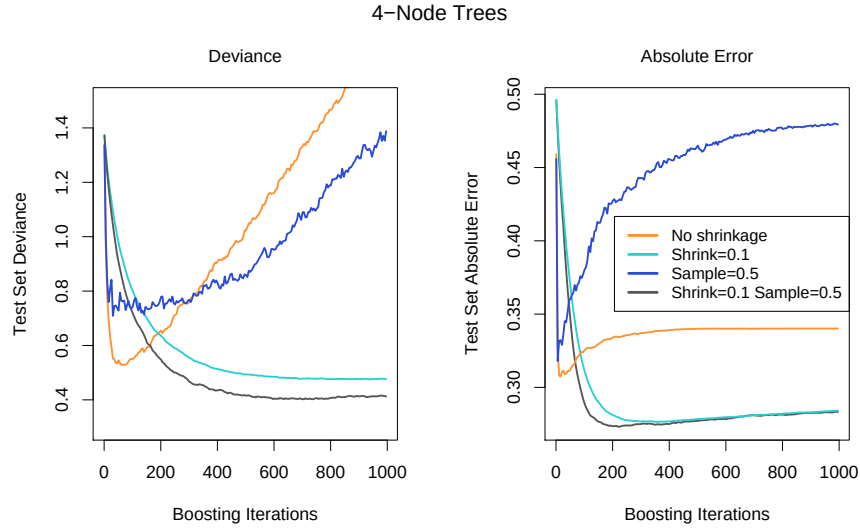


FIGURE 10.12. Test-error curves for the simulated example (10.2), showing the effect of stochasticity. For the curves labeled “Sample=0.5”, a different 50% subsample of the training data was used each time a tree was grown. In the left panel the models were fit by `gbm` using a binomial deviance loss function; in the right-hand panel using square-error loss.

The downside is that we now have four parameters to set: J , M , ν and η . Typically some early explorations determine suitable values for J , ν and η , leaving M as the primary parameter.

10.13 Interpretation

Single decision trees are highly interpretable. The entire model can be completely represented by a simple two-dimensional graphic (binary tree) that is easily visualized. Linear combinations of trees (10.28) lose this important feature, and must therefore be interpreted in a different way.

10.13.1 Relative Importance of Predictor Variables

In data mining applications the input predictor variables are seldom equally relevant. Often only a few of them have substantial influence on the response; the vast majority are irrelevant and could just as well have not been included. It is often useful to learn the relative importance or contribution of each input variable in predicting the response.

For a single decision tree T , Breiman et al. (1984) proposed

$$\mathcal{I}_\ell^2(T) = \sum_{t=1}^{J-1} \hat{\imath}_t^2 I(v(t) = \ell) \quad (10.42)$$

as a measure of relevance for each predictor variable X_ℓ . The sum is over the $J - 1$ internal nodes of the tree. At each such node t , one of the input variables $X_{v(t)}$ is used to partition the region associated with that node into two subregions; within each a separate constant is fit to the response values. The particular variable chosen is the one that gives maximal estimated improvement $\hat{\imath}_t^2$ in squared error risk over that for a constant fit over the entire region. The squared relative importance of variable X_ℓ is the sum of such squared improvements over all internal nodes for which it was chosen as the splitting variable.

This importance measure is easily generalized to additive tree expansions (10.28); it is simply averaged over the trees

$$\mathcal{I}_\ell^2 = \frac{1}{M} \sum_{m=1}^M \mathcal{I}_\ell^2(T_m). \quad (10.43)$$

Due to the stabilizing effect of averaging, this measure turns out to be more reliable than is its counterpart (10.42) for a single tree. Also, because of shrinkage (Section 10.12.1) the masking of important variables by others with which they are highly correlated is much less of a problem. Note that (10.42) and (10.43) refer to *squared* relevance; the actual relevances are their respective square roots. Since these measures are relative, it is customary to assign the largest a value of 100 and then scale the others accordingly. Figure 10.6 shows the relevant importance of the 57 inputs in predicting `spam` versus `email`.

For K -class classification, K separate models $f_k(x)$, $k = 1, 2, \dots, K$ are induced, each consisting of a sum of trees

$$f_k(x) = \sum_{m=1}^M T_{km}(x). \quad (10.44)$$

In this case (10.43) generalizes to

$$\mathcal{I}_{\ell k}^2 = \frac{1}{M} \sum_{m=1}^M \mathcal{I}_\ell^2(T_{km}). \quad (10.45)$$

Here $\mathcal{I}_{\ell k}$ is the relevance of X_ℓ in separating the class k observations from the other classes. The overall relevance of X_ℓ is obtained by averaging over all of the classes

$$\mathcal{I}_\ell^2 = \frac{1}{K} \sum_{k=1}^K \mathcal{I}_{\ell k}^2. \quad (10.46)$$

Figures 10.23 and 10.24 illustrate the use of these averaged and separate relative importances.

10.13.2 Partial Dependence Plots

After the most relevant variables have been identified, the next step is to attempt to understand the nature of the dependence of the approximation $f(X)$ on their joint values. Graphical renderings of the $f(X)$ as a function of its arguments provides a comprehensive summary of its dependence on the joint values of the input variables.

Unfortunately, such visualization is limited to low-dimensional views. We can easily display functions of one or two arguments, either continuous or discrete (or mixed), in a variety of different ways; this book is filled with such displays. Functions of slightly higher dimensions can be plotted by conditioning on particular sets of values of all but one or two of the arguments, producing a *trellis* of plots (Becker et al., 1996).¹

For more than two or three variables, viewing functions of the corresponding higher-dimensional arguments is more difficult. A useful alternative can sometimes be to view a collection of plots, each one of which shows the partial dependence of the approximation $f(X)$ on a selected small subset of the input variables. Although such a collection can seldom provide a comprehensive depiction of the approximation, it can often produce helpful clues, especially when $f(x)$ is dominated by low-order interactions (10.40).

Consider the subvector X_S of $\ell < p$ of the input predictor variables $X^T = (X_1, X_2, \dots, X_p)$, indexed by $\mathcal{S} \subset \{1, 2, \dots, p\}$. Let $\mathcal{C}$ be the complement set, with $\mathcal{S} \cup \mathcal{C} = \{1, 2, \dots, p\}$. A general function $f(X)$ will in principle depend on all of the input variables: $f(X) = f(X_S, X_C)$. One way to define the average or *partial* dependence of $f(X)$ on X_S is

$$f_S(X_S) = E_{X_C} f(X_S, X_C). \quad (10.47)$$

This is a marginal average of f , and can serve as a useful description of the effect of the chosen subset on $f(X)$ when, for example, the variables in X_S do not have strong interactions with those in X_C .

Partial dependence functions can be used to interpret the results of any “black box” learning method. They can be estimated by

$$\bar{f}_S(X_S) = \frac{1}{N} \sum_{i=1}^N f(X_S, x_{iC}), \quad (10.48)$$

where $\{x_{1C}, x_{2C}, \dots, x_{NC}\}$ are the values of X_C occurring in the training data. This requires a pass over the data for each set of joint values of X_S for which $\bar{f}_S(X_S)$ is to be evaluated. This can be computationally intensive,

¹lattice in R.

even for moderately sized data sets. Fortunately with decision trees, $\bar{f}_S(X_S)$ (10.48) can be rapidly computed from the tree itself without reference to the data (Exercise 10.11).

It is important to note that partial dependence functions defined in (10.47) represent the effect of X_S on $f(X)$ after accounting for the (average) effects of the other variables X_C on $f(X)$. They are *not* the effect of X_S on $f(X)$ *ignoring* the effects of X_C . The latter is given by the conditional expectation

$$\tilde{f}_S(X_S) = E(f(X_S, X_C) | X_S), \quad (10.49)$$

and is the best least squares approximation to $f(X)$ by a function of X_S alone. The quantities $\tilde{f}_S(X_S)$ and $\bar{f}_S(X_S)$ will be the same only in the unlikely event that X_S and X_C are independent. For example, if the effect of the chosen variable subset happens to be purely additive,

$$f(X) = h_1(X_S) + h_2(X_C). \quad (10.50)$$

Then (10.47) produces the $h_1(X_S)$ up to an additive constant. If the effect is purely multiplicative,

$$f(X) = h_1(X_S) \cdot h_2(X_C), \quad (10.51)$$

then (10.47) produces $h_1(X_S)$ up to a multiplicative constant factor. On the other hand, (10.49) will not produce $h_1(X_S)$ in either case. In fact, (10.49) can produce strong effects on variable subsets for which $f(X)$ has no dependence at all.

Viewing plots of the partial dependence of the boosted-tree approximation (10.28) on selected variables subsets can help to provide a qualitative description of its properties. Illustrations are shown in Sections 10.8 and 10.14. Owing to the limitations of computer graphics, and human perception, the size of the subsets X_S must be small ($l \approx 1, 2, 3$). There are of course a large number of such subsets, but only those chosen from among the usually much smaller set of highly relevant predictors are likely to be informative. Also, those subsets whose effect on $f(X)$ is approximately additive (10.50) or multiplicative (10.51) will be most revealing.

For K -class classification, there are K separate models (10.44), one for each class. Each one is related to the respective probabilities (10.21) through

$$f_k(X) = \log p_k(X) - \frac{1}{K} \sum_{l=1}^K \log p_l(X). \quad (10.52)$$

Thus each $f_k(X)$ is a monotone increasing function of its respective probability on a logarithmic scale. Partial dependence plots of each respective $f_k(X)$ (10.44) on its most relevant predictors (10.45) can help reveal how the log-odds of realizing that class depend on the respective input variables.

10.14 Illustrations

In this section we illustrate gradient boosting on a number of larger datasets, using different loss functions as appropriate.

10.14.1 *California Housing*

This data set (Pace and Barry, 1997) is available from the Carnegie-Mellon *StatLib* repository². It consists of aggregated data from each of 20,460 neighborhoods (1990 census block groups) in California. The response variable Y is the median house value in each neighborhood measured in units of \$100,000. The predictor variables are demographics such as median income `MedInc`, housing density as reflected by the number of houses `House`, and the average occupancy in each house `AveOccup`. Also included as predictors are the location of each neighborhood (`longitude` and `latitude`), and several quantities reflecting the properties of the houses in the neighborhood: average number of rooms `AveRooms` and bedrooms `AveBedrms`. There are thus a total of eight predictors, all numeric.

We fit a gradient boosting model using the MART procedure, with $J = 6$ terminal nodes, a learning rate (10.41) of $\nu = 0.1$, and the Huber loss criterion for predicting the numeric response. We randomly divided the dataset into a training set (80%) and a test set (20%).

Figure 10.13 shows the average absolute error

$$\text{AAE} = \text{E} |y - \hat{f}_M(x)| \quad (10.53)$$

as a function for number of iterations M on both the training data and test data. The test error is seen to decrease monotonically with increasing M , more rapidly during the early stages and then leveling off to being nearly constant as iterations increase. Thus, the choice of a particular value of M is not critical, as long as it is not too small. This tends to be the case in many applications. The shrinkage strategy (10.41) tends to eliminate the problem of overfitting, especially for larger data sets.

The value of AAE after 800 iterations is 0.31. This can be compared to that of the optimal constant predictor $\text{median}\{y_i\}$ which is 0.89. In terms of more familiar quantities, the squared multiple correlation coefficient of this model is $R^2 = 0.84$. Pace and Barry (1997) use a sophisticated spatial autoregression procedure, where prediction for each neighborhood is based on median house values in nearby neighborhoods, using the other predictors as covariates. Experimenting with transformations they achieved $R^2 = 0.85$, predicting $\log Y$. Using $\log Y$ as the response the corresponding value for gradient boosting was $R^2 = 0.86$.

²<http://lib.stat.cmu.edu>.

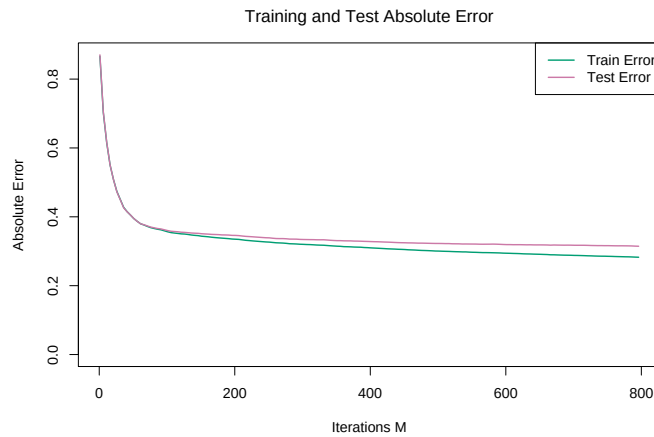


FIGURE 10.13. Average-absolute error as a function of number of iterations for the California housing data.

Figure 10.14 displays the relative variable importances for each of the eight predictor variables. Not surprisingly, median income in the neighborhood is the most relevant predictor. Longitude, latitude, and average occupancy all have roughly half the relevance of income, whereas the others are somewhat less influential.

Figure 10.15 shows single-variable partial dependence plots on the most relevant nonlocation predictors. Note that the plots are not strictly smooth. This is a consequence of using tree-based models. Decision trees produce discontinuous piecewise constant models (10.25). This carries over to sums of trees (10.28), with of course many more pieces. Unlike most of the methods discussed in this book, there is no smoothness constraint imposed on the result. Arbitrarily sharp discontinuities can be modeled. The fact that these curves generally exhibit a smooth trend is because that is what is estimated to best predict the response for this problem. This is often the case.

The hash marks at the base of each plot delineate the deciles of the data distribution of the corresponding variables. Note that here the data density is lower near the edges, especially for larger values. This causes the curves to be somewhat less well determined in those regions. The vertical scales of the plots are the same, and give a visual comparison of the relative importance of the different variables.

The partial dependence of median house value on median income is monotonic increasing, being nearly linear over the main body of data. House value is generally monotonic decreasing with increasing average occupancy, except perhaps for average occupancy rates less than one. Median house

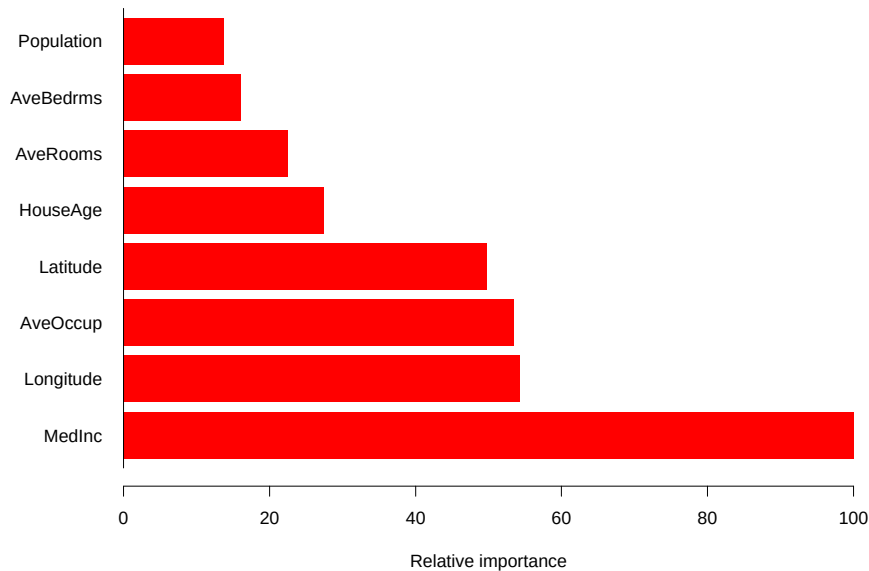


FIGURE 10.14. *Relative importance of the predictors for the California housing data.*

value has a nonmonotonic partial dependence on average number of rooms. It has a minimum at approximately three rooms and is increasing both for smaller and larger values.

Median house value is seen to have a very weak partial dependence on house age that is inconsistent with its importance ranking (Figure 10.14). This suggests that this weak main effect may be masking stronger interaction effects with other variables. Figure 10.16 shows the two-variable partial dependence of housing value on joint values of median age and average occupancy. An interaction between these two variables is apparent. For values of average occupancy greater than two, house value is nearly independent of median age, whereas for values less than two there is a strong dependence on age.

Figure 10.17 shows the two-variable partial dependence of the fitted model on joint values of longitude and latitude, displayed as a shaded contour plot. There is clearly a very strong dependence of median house value on the neighborhood location in California. Note that Figure 10.17 is *not* a plot of house value versus location *ignoring* the effects of the other predictors (10.49). Like all partial dependence plots, it represents the effect of location after accounting for the effects of the other neighborhood and house attributes (10.47). It can be viewed as representing an extra premium one pays for location. This premium is seen to be relatively large near the Pacific coast especially in the Bay Area and Los Angeles–San Diego re-

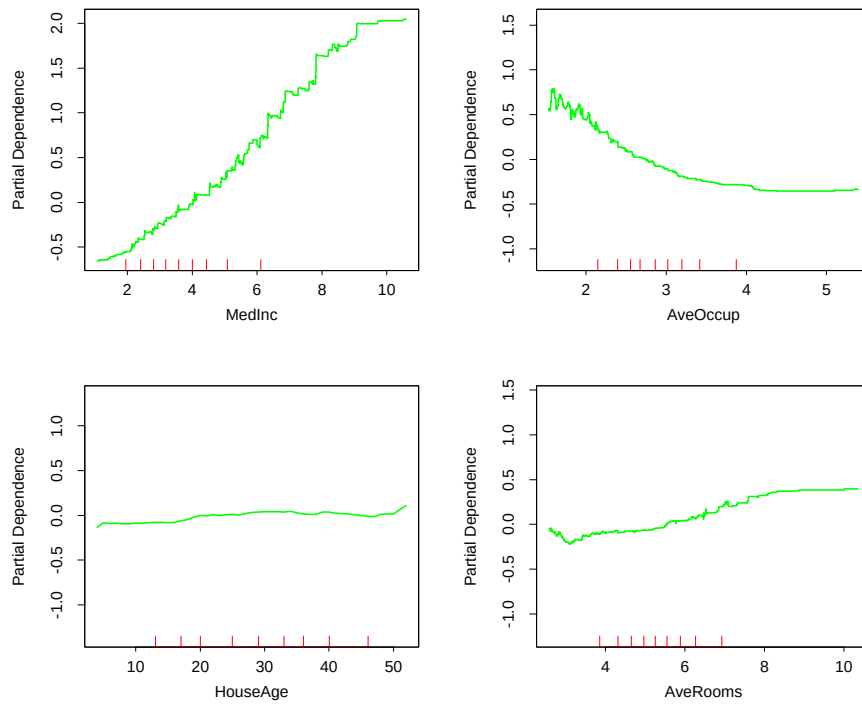


FIGURE 10.15. Partial dependence of housing value on the nonlocation variables for the California housing data. The red ticks at the base of the plot are deciles of the input variables.

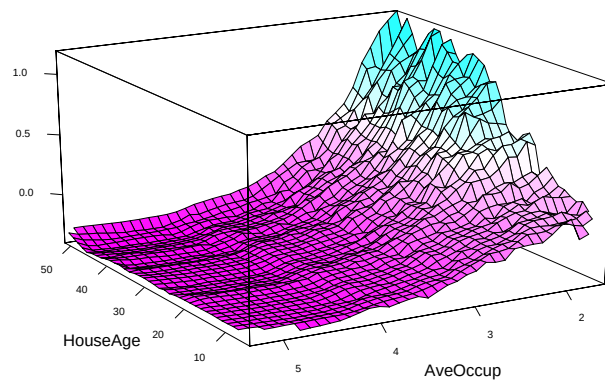


FIGURE 10.16. Partial dependence of house value on median age and average occupancy. There appears to be a strong interaction effect between these two variables.

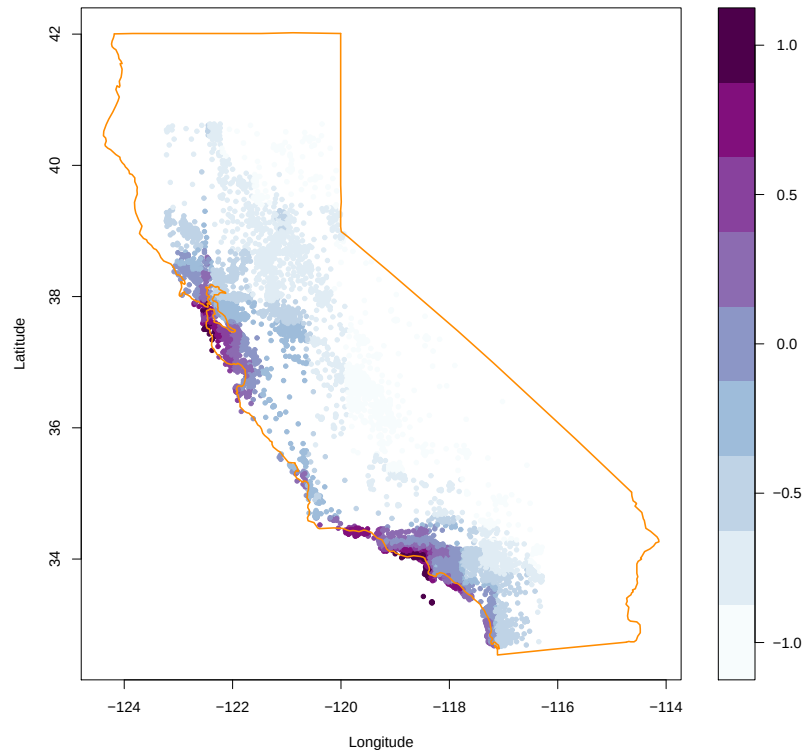


FIGURE 10.17. *Partial dependence of median house value on location in California. One unit is \$100,000, at 1990 prices, and the values plotted are relative to the overall median of \$180,000.*

gions. In the northern, central valley, and southeastern desert regions of California, location costs considerably less.

10.14.2 New Zealand Fish

Plant and animal ecologists use regression models to predict species presence, abundance and richness as a function of environmental variables. Although for many years simple linear and parametric models were popular, recent literature shows increasing interest in more sophisticated models such as generalized additive models (Section 9.1, GAM), multivariate adaptive regression splines (Section 9.4, MARS) and boosted regression trees (Leathwick et al., 2005; Leathwick et al., 2006). Here we model the

presence and abundance of the *Black Oreo Dory*, a marine fish found in the oceanic waters around New Zealand.³

Figure 10.18 shows the locations of 17,000 trawls (deep-water net fishing, with a maximum depth of 2km), and the red points indicate those 2353 trawls for which the Black Oreo was present, one of over a hundred species regularly recorded. The catch size in kg for each species was recorded for each trawl. Along with the species catch, a number of environmental measurements are available for each trawl. These include the average depth of the trawl (**AvgDepth**), and the temperature and salinity of the water. Since the latter two are strongly correlated with depth, Leathwick et al. (2006) derived instead **TempResid** and **SalResid**, the residuals obtained when these two measures are adjusted for depth (via separate non-parametric regressions). **SSTGrad** is a measure of the gradient of the sea surface temperature, and **Chla** is a broad indicator of ecosystem productivity via satellite-image measurements. **SusPartMatter** provides a measure of suspended particulate matter, particularly in coastal waters, and is also satellite derived.

The goal of this analysis is to estimate the probability of finding Black Oreo in a trawl, as well as the expected catch size, standardized to take into account the effects of variation in trawl speed and distance, as well as the mesh size of the trawl net. The authors used logistic regression for estimating the probability. For the catch size, it might seem natural to assume a Poisson distribution and model the log of the mean count, but this is often not appropriate because of the excessive number of zeros. Although specialized approaches have been developed, such as the *zero-inflated* Poisson (Lambert, 1992), they chose a simpler approach. If Y is the (non-negative) catch size,

$$E(Y|X) = E(Y|Y > 0, X) \cdot \Pr(Y > 0|X). \quad (10.54)$$

The second term is estimated by the logistic regression, and the first term can be estimated using only the 2353 trawls with a positive catch.

For the logistic regression the authors used a gradient boosted model (GBM)⁴ with binomial deviance loss function, depth-10 trees, and a shrinkage factor $\nu = 0.025$. For the positive-catch regression, they modeled $\log(Y)$ using a GBM with squared-error loss (also depth-10 trees, but $\nu = 0.01$), and un-logged the predictions. In both cases they used 10-fold cross-validation for selecting the number of terms, as well as the shrinkage factor.

³The models, data, and maps shown here were kindly provided by Dr John Leathwick of the National Institute of Water and Atmospheric Research in New Zealand, and Dr Jane Elith, School of Botany, University of Melbourne. The collection of the research trawl data took place from 1979–2005, and was funded by the New Zealand Ministry of Fisheries.

⁴Version 1.5-7 of package **gbm** in R, ver. 2.2.0.

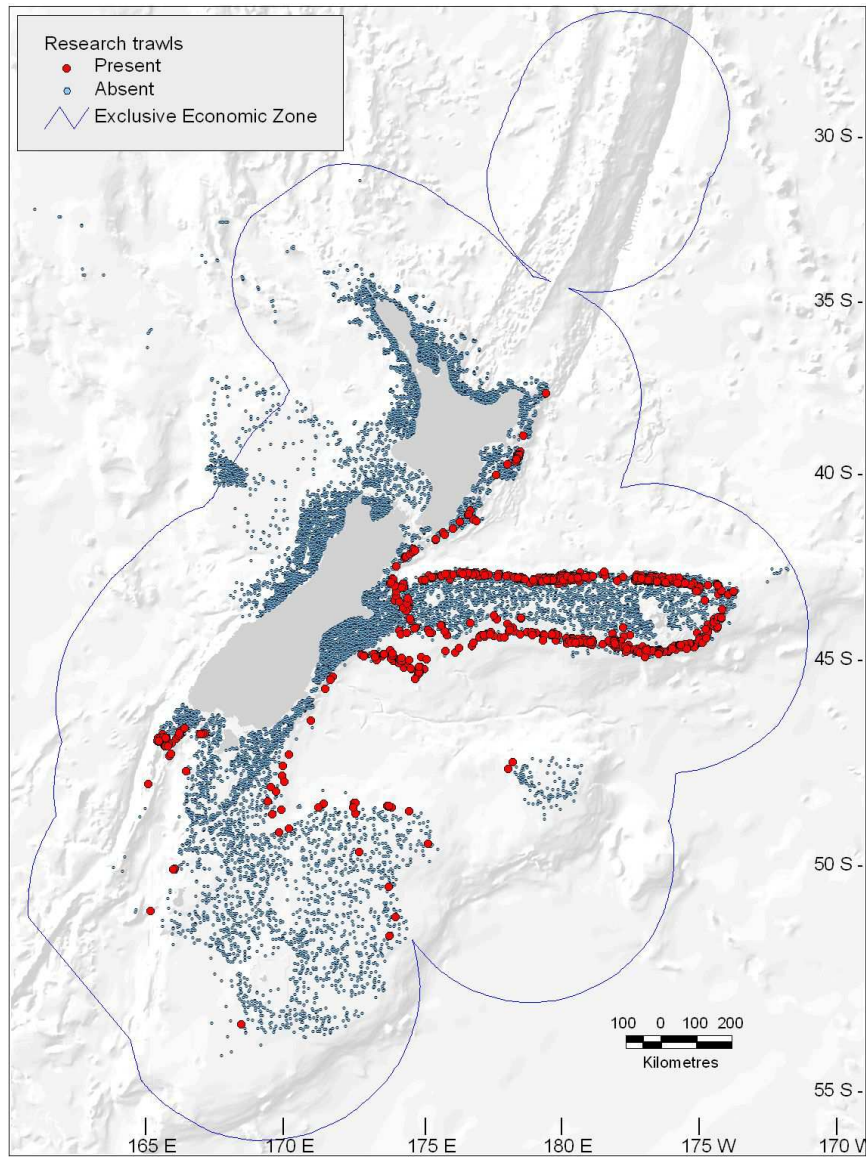


FIGURE 10.18. Map of New Zealand and its surrounding exclusive economic zone, showing the locations of 17,000 trawls (small blue dots) taken between 1979 and 2005. The red points indicate trawls for which the species Black Oreo Dory were present.

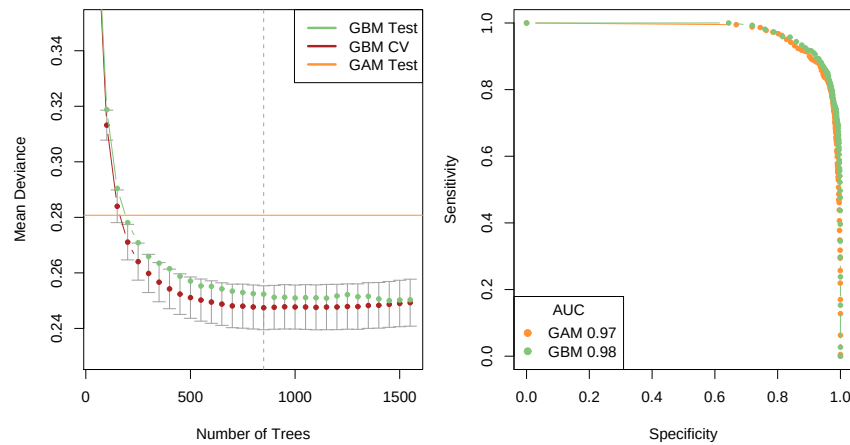


FIGURE 10.19. The left panel shows the mean deviance as a function of the number of trees for the GBM logistic regression model fit to the presence/absence data. Shown are 10-fold cross-validation on the training data (and $1 \times$ s.e. bars), and test deviance on the test data. Also shown for comparison is the test deviance using a GAM model with 8 df for each term. The right panel shows ROC curves on the test data for the chosen GBM model (vertical line in left plot) and the GAM model.

Figure 10.19 (left panel) shows the mean binomial deviance for the sequence of GBM models, both for 10-fold CV and test data. There is a modest improvement over the performance of a GAM model, fit using smoothing splines with 8 degrees-of-freedom (df) per term. The right panel shows the ROC curves (see Section 9.2.5) for both models, which measures predictive performance. From this point of view, the performance looks very similar, with GBM perhaps having a slight edge as summarized by the AUC (area under the curve). At the point of equal sensitivity/specificity, GBM achieves 91%, and GAM 90%.

Figure 10.20 summarizes the contributions of the variables in the logistic GBM fit. We see that there is a well-defined depth range over which Black Oreo are caught, with much more frequent capture in colder waters. We do not give details of the quantitative catch model; the important variables were much the same.

All the predictors used in these models are available on a fine geographical grid; in fact they were derived from environmental atlases, satellite images and the like—see Leathwick et al. (2006) for details. This also means that predictions can be made on this grid, and imported into GIS mapping systems. Figure 10.21 shows prediction maps for both presence and catch size, with both standardized to a common set of trawl conditions; since the predictors vary in a continuous fashion with geographical location, so do the predictions.

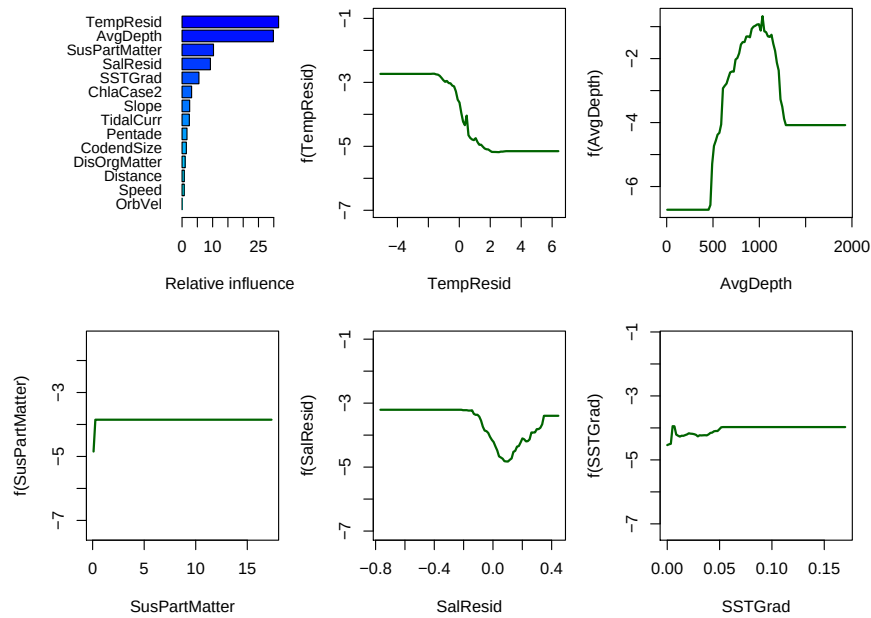


FIGURE 10.20. The top-left panel shows the relative influence computed from the GBM logistic regression model. The remaining panels show the partial dependence plots for the leading five variables, all plotted on the same scale for comparison.

Because of their ability to model interactions and automatically select variables, as well as robustness to outliers and missing data, GBM models are rapidly gaining popularity in this data-rich and enthusiastic community.

10.14.3 Demographics Data

In this section we illustrate gradient boosting on a multiclass classification problem, using MART. The data come from 9243 questionnaires filled out by shopping mall customers in the San Francisco Bay Area (Impact Resources, Inc., Columbus, OH). Among the questions are 14 concerning demographics. For this illustration the goal is to predict occupation using the other 13 variables as predictors, and hence identify demographic variables that discriminate between different occupational categories. We randomly divided the data into a training set (80%) and test set (20%), and used $J = 6$ node trees with a learning rate $\nu = 0.1$.

Figure 10.22 shows the $K = 9$ occupation class values along with their corresponding error rates. The overall error rate is 42.5%, which can be compared to the null rate of 69% obtained by predicting the most numerous

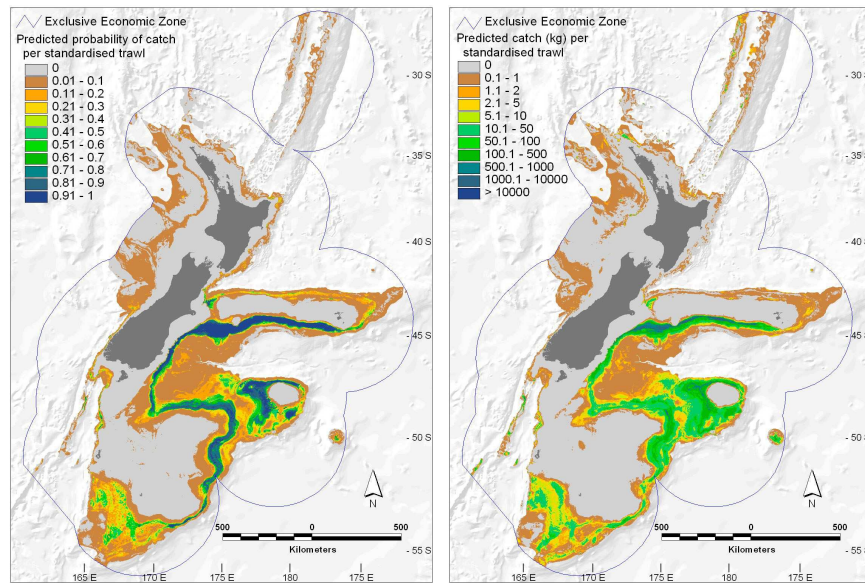


FIGURE 10.21. Geological prediction maps of the presence probability (left map) and catch size (right map) obtained from the gradient boosted models.

class **Prof/Man** (Professional/Managerial). The four best predicted classes are seen to be **Retired**, **Student**, **Prof/Man**, and **Homemaker**.

Figure 10.23 shows the relative predictor variable importances as averaged over all classes (10.46). Figure 10.24 displays the individual relative importance distributions (10.45) for each of the four best predicted classes. One sees that the most relevant predictors are generally different for each respective class. An exception is **age** which is among the three most relevant for predicting **Retired**, **Student**, and **Prof/Man**.

Figure 10.25 shows the partial dependence of the log-odds (10.52) on **age** for these three classes. The abscissa values are ordered codes for respective equally spaced age intervals. One sees that after accounting for the contributions of the other variables, the odds of being retired are higher for older people, whereas the opposite is the case for being a student. The odds of being professional/managerial are highest for middle-aged people. These results are of course not surprising. They illustrate that inspecting partial dependences separately for each class can lead to sensible results.

Bibliographic Notes

Schapire (1990) developed the first simple boosting procedure in the PAC learning framework (Valiant, 1984; Kearns and Vazirani, 1994). Schapire

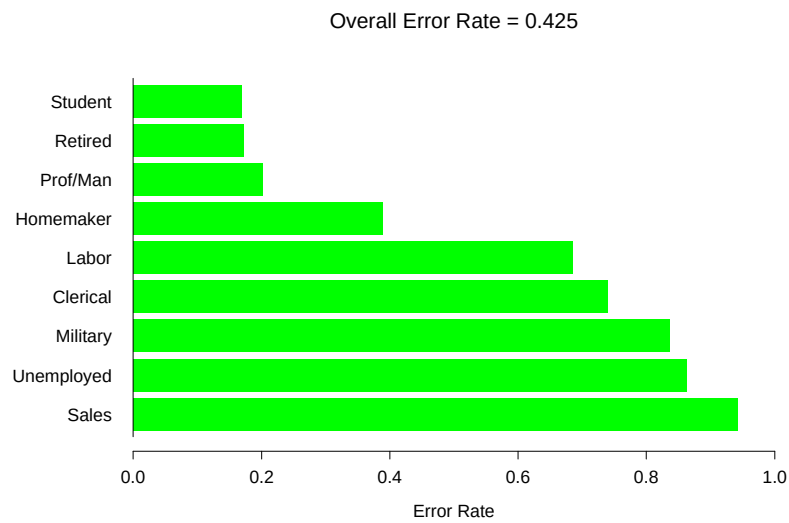


FIGURE 10.22. Error rate for each occupation in the demographics data.

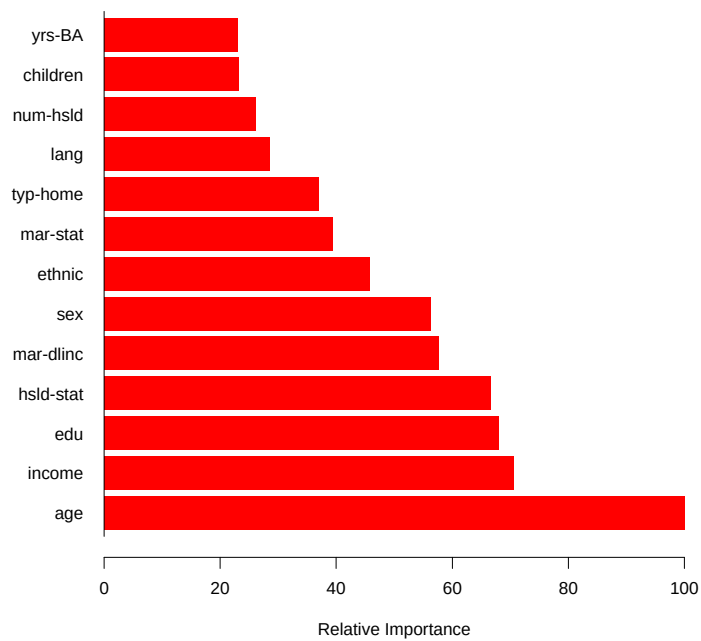


FIGURE 10.23. Relative importance of the predictors as averaged over all classes for the demographics data.

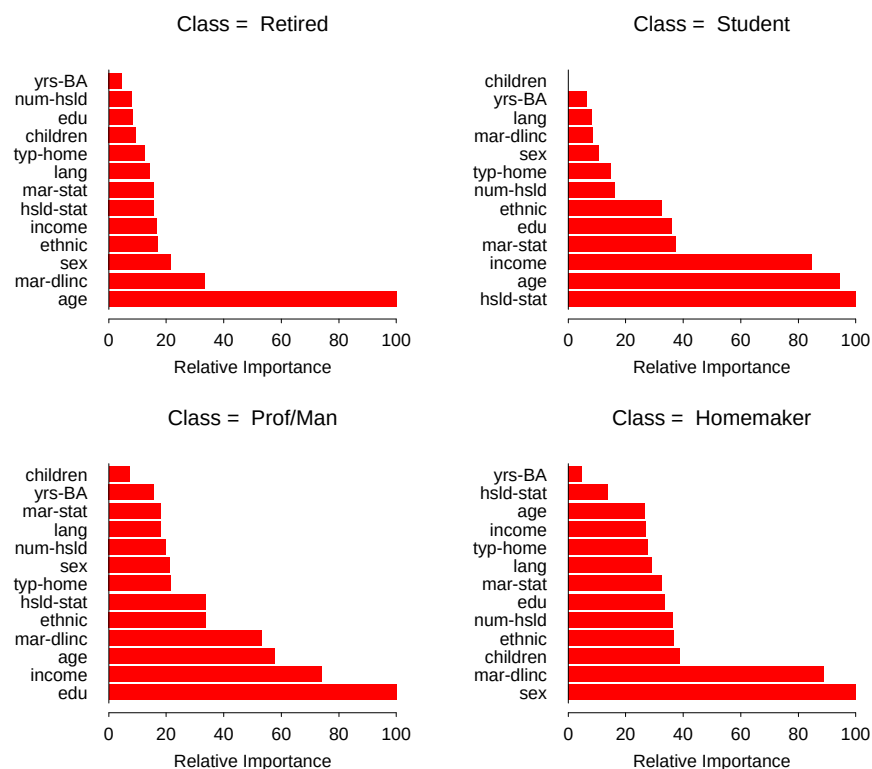


FIGURE 10.24. Predictor variable importances separately for each of the four classes with lowest error rate for the demographics data.

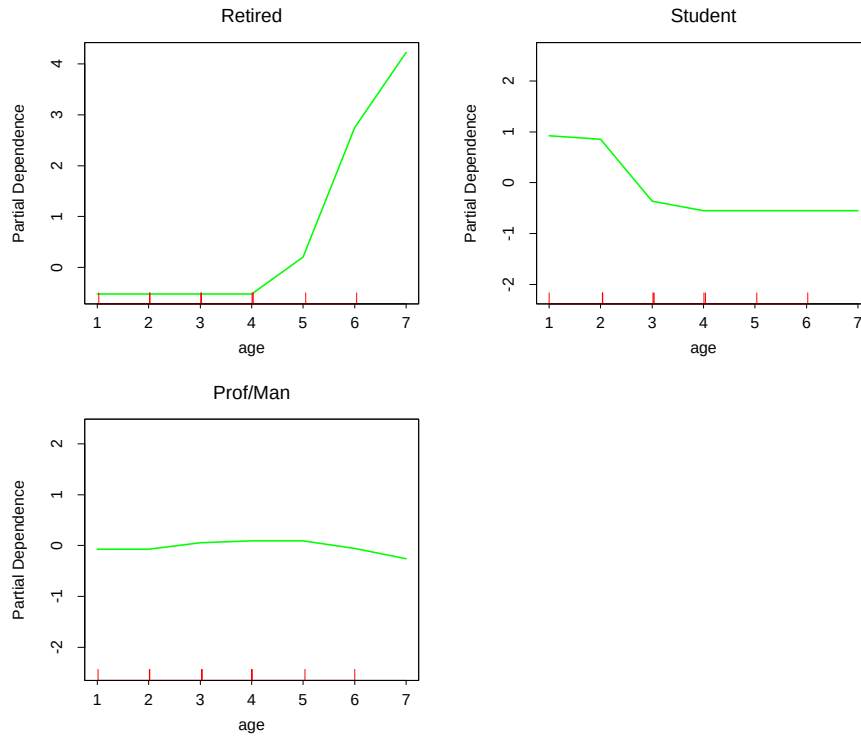


FIGURE 10.25. Partial dependence of the odds of three different occupations on age, for the demographics data.

showed that a *weak learner* could always improve its performance by training two additional classifiers on filtered versions of the input data stream. A weak learner is an algorithm for producing a two-class classifier with performance guaranteed (with high probability) to be significantly better than a coin-flip. After learning an initial classifier G_1 on the first N training points,

- G_2 is learned on a new sample of N points, half of which are misclassified by G_1 ;
- G_3 is learned on N points for which G_1 and G_2 disagree;
- the boosted classifier is $G_B = \text{majority vote}(G_1, G_2, G_3)$.

Schapire's “Strength of Weak Learnability” theorem proves that G_B has improved performance over G_1 .

Freund (1995) proposed a “boost by majority” variation which combined many weak learners simultaneously and improved the performance of the simple boosting algorithm of Schapire. The theory supporting both of these

algorithms requires the weak learner to produce a classifier with a fixed error rate. This led to the more adaptive and realistic AdaBoost (Freund and Schapire, 1996a) and its offspring, where this assumption was dropped.

Freund and Schapire (1996a) and Schapire and Singer (1999) provide some theory to support their algorithms, in the form of upper bounds on generalization error. This theory has evolved in the computational learning community, initially based on the concepts of PAC learning. Other theories attempting to explain boosting come from game theory (Freund and Schapire, 1996b; Breiman, 1999; Breiman, 1998), and VC theory (Schapire et al., 1998). The bounds and the theory associated with the AdaBoost algorithms are interesting, but tend to be too loose to be of practical importance. In practice, boosting achieves results far more impressive than the bounds would imply. Schapire (2002) and Meir and Rätsch (2003) give useful overviews more recent than the first edition of this book.

Friedman et al. (2000) and Friedman (2001) form the basis for our exposition in this chapter. Friedman et al. (2000) analyze AdaBoost statistically, derive the exponential criterion, and show that it estimates the log-odds of the class probability. They propose additive tree models, the right-sized trees and ANOVA representation of Section 10.11, and the multiclass logit formulation. Friedman (2001) developed gradient boosting and shrinkage for classification and regression, while Friedman (1999) explored stochastic variants of boosting. Mason et al. (2000) also embraced a gradient approach to boosting. As the published discussions of Friedman et al. (2000) shows, there is some controversy about how and why boosting works.

Since the publication of the first edition of this book, these debates have continued, and spread into the statistical community with a series of papers on consistency of boosting (Jiang, 2004; Lugosi and Vayatis, 2004; Zhang and Yu, 2005; Bartlett and Traskin, 2007). Mease and Wyner (2008), through a series of simulation examples, challenge some of our interpretations of boosting; our response (Friedman et al., 2008a) puts most of these objections to rest. A recent survey by Bühlmann and Hothorn (2007) supports our approach to boosting.

Exercises

Ex. 10.1 Derive expression (10.12) for the update parameter in AdaBoost.

Ex. 10.2 Prove result (10.16), that is, the minimizer of the population version of the AdaBoost criterion, is one-half of the log odds.

Ex. 10.3 Show that the marginal average (10.47) recovers additive and multiplicative functions (10.50) and (10.51), while the conditional expectation (10.49) does not.

Ex. 10.4

- (a) Write a program implementing AdaBoost with trees.
- (b) Redo the computations for the example of Figure 10.2. Plot the training error as well as test error, and discuss its behavior.
- (c) Investigate the number of iterations needed to make the test error finally start to rise.
- (d) Change the setup of this example as follows: define two classes, with the features in Class 1 being $X_1, X_2, \dots, X_{10}$, standard independent Gaussian variates. In Class 2, the features $X_1, X_2, \dots, X_{10}$ are also standard independent Gaussian, but conditioned on the event $\sum_j X_j^2 > 12$. Now the classes have significant overlap in feature space. Repeat the AdaBoost experiments as in Figure 10.2 and discuss the results.

Ex. 10.5 *Multiclass exponential loss* (Zhu et al., 2005). For a K -class classification problem, consider the coding $Y = (Y_1, \dots, Y_K)^T$ with

$$Y_k = \begin{cases} 1, & \text{if } G = \mathcal{G}_k \\ -\frac{1}{K-1}, & \text{otherwise.} \end{cases} \quad (10.55)$$

Let $f = (f_1, \dots, f_K)^T$ with $\sum_{k=1}^K f_k = 0$, and define

$$L(Y, f) = \exp\left(-\frac{1}{K} Y^T f\right). \quad (10.56)$$

- (a) Using Lagrange multipliers, derive the population minimizer f^* of $L(Y, f)$, subject to the zero-sum constraint, and relate these to the class probabilities.
- (b) Show that a multiclass boosting using this loss function leads to a reweighting algorithm similar to Adaboost, as in Section 10.4.

Ex. 10.6 *McNemar test* (Agresti, 1996). We report the test error rates on the spam data to be 5.5% for a generalized additive model (GAM), and 4.5% for gradient boosting (GBM), with a test sample of size 1536.

- (a) Show that the standard error of these estimates is about 0.6%.

Since the same test data are used for both methods, the error rates are correlated, and we cannot perform a two-sample t-test. We can compare the methods directly on each test observation, leading to the summary

GAM	GBM	
	Correct	Error
Correct	1434	18
Error	33	51

The McNemar test focuses on the discordant errors, 33 vs. 18.

- (b) Conduct a test to show that GAM makes significantly more errors than gradient boosting, with a two-sided p-value of 0.036.

Ex. 10.7 Derive expression (10.32).

Ex. 10.8 Consider a K -class problem where the targets y_{ik} are coded as 1 if observation i is in class k and zero otherwise. Suppose we have a current model $f_k(x)$, $k = 1, \dots, K$, with $\sum_{k=1}^K f_k(x) = 0$ (see (10.21) in Section 10.6). We wish to update the model for observations in a region R in predictor space, by adding constants $f_k(x) + \gamma_k$, with $\gamma_K = 0$.

- (a) Write down the multinomial log-likelihood for this problem, and its first and second derivatives.
- (b) Using only the diagonal of the Hessian matrix in (1), and starting from $\gamma_k = 0 \forall k$, show that a one-step approximate Newton update for γ_k is

$$\gamma_k^1 = \frac{\sum_{x_i \in R} (y_{ik} - p_{ik})}{\sum_{x_i \in R} p_{ik}(1 - p_{ik})}, \quad k = 1, \dots, K-1, \quad (10.57)$$

where $p_{ik} = \exp(f_k(x_i)) / \exp(\sum_{\ell=1}^K f_\ell(x_i))$.

- (c) We prefer our update to sum to zero, as the current model does. Using symmetry arguments, show that

$$\hat{\gamma}_k = \frac{K-1}{K} (\gamma_k^1 - \frac{1}{K} \sum_{\ell=1}^K \gamma_\ell^1), \quad k = 1, \dots, K \quad (10.58)$$

is an appropriate update, where γ_k^1 is defined as in (10.57) for all $k = 1, \dots, K$.

Ex. 10.9 Consider a K -class problem where the targets y_{ik} are coded as 1 if observation i is in class k and zero otherwise. Using the multinomial deviance loss function (10.22) and the symmetric logistic transform, use the arguments leading to the gradient boosting Algorithm 10.3 to derive Algorithm 10.4. *Hint:* See exercise 10.8 for step 2(b)iii.

Ex. 10.10 Show that for $K = 2$ class classification, only one tree needs to be grown at each gradient-boosting iteration.

Ex. 10.11 Show how to compute the partial dependence function $f_S(X_S)$ in (10.47) efficiently.

Ex. 10.12 Referring to (10.49), let $\mathcal{S} = \{1\}$ and $\mathcal{C} = \{2\}$, with $f(X_1, X_2) = X_1$. Assume X_1 and X_2 are bivariate Gaussian, each with mean zero, variance one, and $E(X_1 X_2) = \rho$. Show that $E(f(X_1, X_2) | X_2) = \rho X_2$, even though f is not a function of X_2 .

Algorithm 10.4 *Gradient Boosting for K -class Classification.*

1. Initialize $f_{k0}(x) = 0$, $k = 1, 2, \dots, K$.

2. For $m=1$ to M :

(a) Set

$$p_k(x) = \frac{e^{f_k(x)}}{\sum_{\ell=1}^K e^{f_\ell(x)}}, \quad k = 1, 2, \dots, K.$$

(b) For $k = 1$ to K :

- i. Compute $r_{ikm} = y_{ik} - p_k(x_i)$, $i = 1, 2, \dots, N$.
- ii. Fit a regression tree to the targets r_{ikm} , $i = 1, 2, \dots, N$, giving terminal regions R_{jkm} , $j = 1, 2, \dots, J_m$.
- iii. Compute

$$\gamma_{jkm} = \frac{K-1}{K} \frac{\sum_{x_i \in R_{jkm}} r_{ikm}}{\sum_{x_i \in R_{jkm}} |r_{ikm}|(1 - |r_{ikm}|)}, \quad j = 1, 2, \dots, J_m.$$

- iv. Update $f_{km}(x) = f_{k,m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jkm} I(x \in R_{jkm})$.

3. Output $\hat{f}_k(x) = f_{kM}(x)$, $k = 1, 2, \dots, K$.

11

Neural Networks

11.1 Introduction

In this chapter we describe a class of learning methods that was developed separately in different fields—statistics and artificial intelligence—based on essentially identical models. The central idea is to extract linear combinations of the inputs as derived features, and then model the target as a nonlinear function of these features. The result is a powerful learning method, with widespread applications in many fields. We first discuss the projection pursuit model, which evolved in the domain of semiparametric statistics and smoothing. The rest of the chapter is devoted to neural network models.

11.2 Projection Pursuit Regression

As in our generic supervised learning problem, assume we have an input vector X with p components, and a target Y . Let ω_m , $m = 1, 2, \dots, M$, be unit p -vectors of unknown parameters. The projection pursuit regression (PPR) model has the form

$$f(X) = \sum_{m=1}^M g_m(\omega_m^T X). \quad (11.1)$$

This is an additive model, but in the derived features $V_m = \omega_m^T X$ rather than the inputs themselves. The functions g_m are unspecified and are esti-

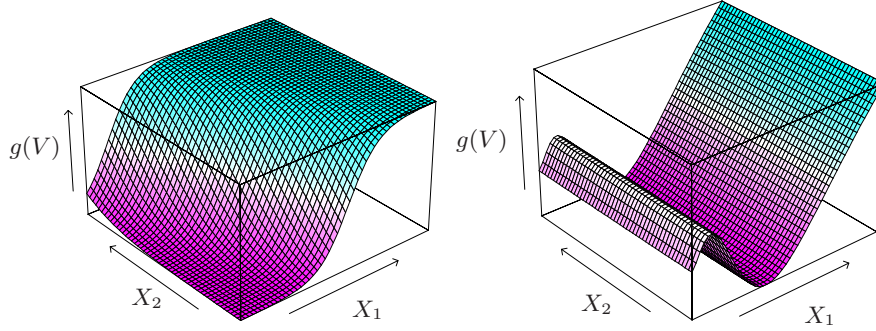


FIGURE 11.1. Perspective plots of two ridge functions.
 (Left:) $g(V) = 1/[1 + \exp(-5(V - 0.5))]$, where $V = (X_1 + X_2)/\sqrt{2}$.
 (Right:) $g(V) = (V + 0.1) \sin(1/(V/3 + 0.1))$, where $V = X_1$.

ated along with the directions ω_m using some flexible smoothing method (see below).

The function $g_m(\omega_m^T X)$ is called a *ridge function* in $\mathbb{R}^p$. It varies only in the direction defined by the vector ω_m . The scalar variable $V_m = \omega_m^T X$ is the projection of X onto the unit vector ω_m , and we seek ω_m so that the model fits well, hence the name “projection pursuit.” Figure 11.1 shows some examples of ridge functions. In the example on the left $\omega = (1/\sqrt{2})(1, 1)^T$, so that the function only varies in the direction $X_1 + X_2$. In the example on the right, $\omega = (1, 0)$.

The PPR model (11.1) is very general, since the operation of forming nonlinear functions of linear combinations generates a surprisingly large class of models. For example, the product $X_1 \cdot X_2$ can be written as $[(X_1 + X_2)^2 - (X_1 - X_2)^2]/4$, and higher-order products can be represented similarly.

In fact, if M is taken arbitrarily large, for appropriate choice of g_m the PPR model can approximate any continuous function in $\mathbb{R}^p$ arbitrarily well. Such a class of models is called a *universal approximator*. However this generality comes at a price. Interpretation of the fitted model is usually difficult, because each input enters into the model in a complex and multifaceted way. As a result, the PPR model is most useful for prediction, and not very useful for producing an understandable model for the data. The $M = 1$ model, known as the *single index model* in econometrics, is an exception. It is slightly more general than the linear regression model, and offers a similar interpretation.

How do we fit a PPR model, given training data (x_i, y_i) , $i = 1, 2, \dots, N$? We seek the approximate minimizers of the error function

$$\sum_{i=1}^N \left[y_i - \sum_{m=1}^M g_m(\omega_m^T x_i) \right]^2 \quad (11.2)$$

over functions g_m and direction vectors ω_m , $m = 1, 2, \dots, M$. As in other smoothing problems, we need either explicitly or implicitly to impose complexity constraints on the g_m , to avoid overfit solutions.

Consider just one term ($M = 1$, and drop the subscript). Given the direction vector ω , we form the derived variables $v_i = \omega^T x_i$. Then we have a one-dimensional smoothing problem, and we can apply any scatterplot smoother, such as a smoothing spline, to obtain an estimate of g .

On the other hand, given g , we want to minimize (11.2) over ω . A Gauss–Newton search is convenient for this task. This is a quasi-Newton method, in which the part of the Hessian involving the second derivative of g is discarded. It can be simply derived as follows. Let ω_{old} be the current estimate for ω . We write

$$g(\omega^T x_i) \approx g(\omega_{\text{old}}^T x_i) + g'(\omega_{\text{old}}^T x_i)(\omega - \omega_{\text{old}})^T x_i \quad (11.3)$$

to give

$$\sum_{i=1}^N [y_i - g(\omega^T x_i)]^2 \approx \sum_{i=1}^N g'(\omega_{\text{old}}^T x_i)^2 \left[\left(\omega_{\text{old}}^T x_i + \frac{y_i - g(\omega_{\text{old}}^T x_i)}{g'(\omega_{\text{old}}^T x_i)} \right) - \omega^T x_i \right]^2. \quad (11.4)$$

To minimize the right-hand side, we carry out a least squares regression with target $\omega_{\text{old}}^T x_i + (y_i - g(\omega_{\text{old}}^T x_i))/g'(\omega_{\text{old}}^T x_i)$ on the input x_i , with weights $g'(\omega_{\text{old}}^T x_i)^2$ and no intercept (bias) term. This produces the updated coefficient vector ω_{new} .

These two steps, estimation of g and ω , are iterated until convergence. With more than one term in the PPR model, the model is built in a forward stage-wise manner, adding a pair (ω_m, g_m) at each stage.

There are a number of implementation details.

- Although any smoothing method can in principle be used, it is convenient if the method provides derivatives. Local regression and smoothing splines are convenient.
- After each step the g_m 's from previous steps can be readjusted using the backfitting procedure described in Chapter 9. While this may lead ultimately to fewer terms, it is not clear whether it improves prediction performance.
- Usually the ω_m are not readjusted (partly to avoid excessive computation), although in principle they could be as well.
- The number of terms M is usually estimated as part of the forward stage-wise strategy. The model building stops when the next term does not appreciably improve the fit of the model. Cross-validation can also be used to determine M .

There are many other applications, such as density estimation (Friedman et al., 1984; Friedman, 1987), where the projection pursuit idea can be used. In particular, see the discussion of ICA in Section 14.7 and its relationship with exploratory projection pursuit. However the projection pursuit regression model has not been widely used in the field of statistics, perhaps because at the time of its introduction (1981), its computational demands exceeded the capabilities of most readily available computers. But it does represent an important intellectual advance, one that has blossomed in its reincarnation in the field of neural networks, the topic of the rest of this chapter.

11.3 Neural Networks

The term *neural network* has evolved to encompass a large class of models and learning methods. Here we describe the most widely used “vanilla” neural net, sometimes called the single hidden layer back-propagation network, or single layer perceptron. There has been a great deal of *hype* surrounding neural networks, making them seem magical and mysterious. As we make clear in this section, they are just nonlinear statistical models, much like the projection pursuit regression model discussed above.

A neural network is a two-stage regression or classification model, typically represented by a *network diagram* as in Figure 11.2. This network applies both to regression or classification. For regression, typically $K = 1$ and there is only one output unit Y_1 at the top. However, these networks can handle multiple quantitative responses in a seamless fashion, so we will deal with the general case.

For K -class classification, there are K units at the top, with the k th unit modeling the probability of class k . There are K target measurements Y_k , $k = 1, \dots, K$, each being coded as a 0 – 1 variable for the k th class.

Derived features Z_m are created from linear combinations of the inputs, and then the target Y_k is modeled as a function of linear combinations of the Z_m ,

$$\begin{aligned} Z_m &= \sigma(\alpha_{0m} + \alpha_m^T X), \quad m = 1, \dots, M, \\ T_k &= \beta_{0k} + \beta_k^T Z, \quad k = 1, \dots, K, \\ f_k(X) &= g_k(T), \quad k = 1, \dots, K, \end{aligned} \tag{11.5}$$

where $Z = (Z_1, Z_2, \dots, Z_M)$, and $T = (T_1, T_2, \dots, T_K)$.

The activation function $\sigma(v)$ is usually chosen to be the *sigmoid* $\sigma(v) = 1/(1 + e^{-v})$; see Figure 11.3 for a plot of $1/(1 + e^{-v})$. Sometimes Gaussian radial basis functions (Chapter 6) are used for the $\sigma(v)$, producing what is known as a *radial basis function network*.

Neural network diagrams like Figure 11.2 are sometimes drawn with an additional *bias* unit feeding into every unit in the hidden and output layers.

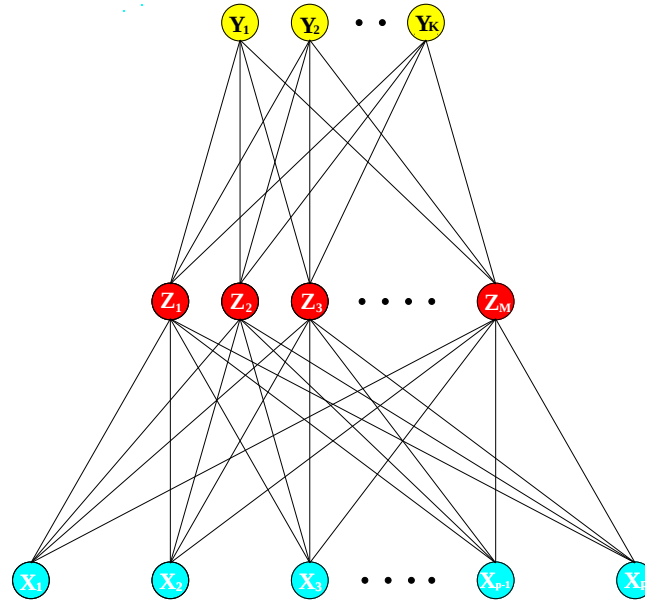


FIGURE 11.2. Schematic of a single hidden layer, feed-forward neural network.

Thinking of the constant “1” as an additional input feature, this bias unit captures the intercepts α_{0m} and β_{0k} in model (11.5).

The output function $g_k(T)$ allows a final transformation of the vector of outputs T . For regression we typically choose the identity function $g_k(T) = T_k$. Early work in K -class classification also used the identity function, but this was later abandoned in favor of the *softmax* function

$$g_k(T) = \frac{e^{T_k}}{\sum_{\ell=1}^K e^{T_\ell}}. \quad (11.6)$$

This is of course exactly the transformation used in the multilogit model (Section 4.4), and produces positive estimates that sum to one. In Section 4.2 we discuss other problems with linear activation functions, in particular potentially severe masking effects.

The units in the middle of the network, computing the derived features Z_m , are called *hidden units* because the values Z_m are not directly observed. In general there can be more than one hidden layer, as illustrated in the example at the end of this chapter. We can think of the Z_m as a basis expansion of the original inputs X ; the neural network is then a standard linear model, or linear multilogit model, using these transformations as inputs. There is, however, an important enhancement over the basis-expansion techniques discussed in Chapter 5; here the parameters of the basis functions are learned from the data.

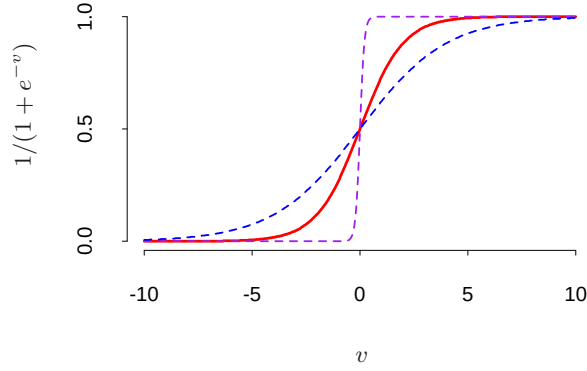


FIGURE 11.3. Plot of the sigmoid function $\sigma(v) = 1/(1+\exp(-v))$ (red curve), commonly used in the hidden layer of a neural network. Included are $\sigma(sv)$ for $s = \frac{1}{2}$ (blue curve) and $s = 10$ (purple curve). The scale parameter s controls the activation rate, and we can see that large s amounts to a hard activation at $v = 0$. Note that $\sigma(s(v - v_0))$ shifts the activation threshold from 0 to v_0 .

Notice that if σ is the identity function, then the entire model collapses to a linear model in the inputs. Hence a neural network can be thought of as a nonlinear generalization of the linear model, both for regression and classification. By introducing the nonlinear transformation σ , it greatly enlarges the class of linear models. In Figure 11.3 we see that the rate of activation of the sigmoid depends on the norm of α_m , and if $\|\alpha_m\|$ is very small, the unit will indeed be operating in the *linear part* of its activation function.

Notice also that the neural network model with one hidden layer has exactly the same form as the projection pursuit model described above. The difference is that the PPR model uses nonparametric functions $g_m(v)$, while the neural network uses a far simpler function based on $\sigma(v)$, with three free parameters in its argument. In detail, viewing the neural network model as a PPR model, we identify

$$\begin{aligned} g_m(\omega_m^T X) &= \beta_m \sigma(\alpha_{0m} + \alpha_m^T X) \\ &= \beta_m \sigma(\alpha_{0m} + \|\alpha_m\|(\omega_m^T X)), \end{aligned} \quad (11.7)$$

where $\omega_m = \alpha_m / \|\alpha_m\|$ is the m th unit-vector. Since $\sigma_{\beta, \alpha_0, s}(v) = \beta \sigma(\alpha_0 + sv)$ has lower complexity than a more general nonparametric $g(v)$, it is not surprising that a neural network might use 20 or 100 such functions, while the PPR model typically uses fewer terms ($M = 5$ or 10 , for example).

Finally, we note that the name “neural networks” derives from the fact that they were first developed as models for the human brain. Each unit represents a neuron, and the connections (links in Figure 11.2) represent synapses. In early models, the neurons fired when the total signal passed to that unit exceeded a certain threshold. In the model above, this corresponds

to use of a step function for $\sigma(Z)$ and $g_m(T)$. Later the neural network was recognized as a useful tool for nonlinear statistical modeling, and for this purpose the step function is not smooth enough for optimization. Hence the step function was replaced by a smoother threshold function, the sigmoid in Figure 11.3.

11.4 Fitting Neural Networks

The neural network model has unknown parameters, often called *weights*, and we seek values for them that make the model fit the training data well. We denote the complete set of weights by θ , which consists of

$$\begin{aligned} \{\alpha_{0m}, \alpha_m; m = 1, 2, \dots, M\} & \quad M(p+1) \text{ weights,} \\ \{\beta_{0k}, \beta_k; k = 1, 2, \dots, K\} & \quad K(M+1) \text{ weights.} \end{aligned} \quad (11.8)$$

For regression, we use sum-of-squared errors as our measure of fit (error function)

$$R(\theta) = \sum_{k=1}^K \sum_{i=1}^N (y_{ik} - f_k(x_i))^2. \quad (11.9)$$

For classification we use either squared error or cross-entropy (deviance):

$$R(\theta) = - \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log f_k(x_i), \quad (11.10)$$

and the corresponding classifier is $G(x) = \operatorname{argmax}_k f_k(x)$. With the softmax activation function and the cross-entropy error function, the neural network model is exactly a linear logistic regression model in the hidden units, and all the parameters are estimated by maximum likelihood.

Typically we don't want the global minimizer of $R(\theta)$, as this is likely to be an overfit solution. Instead some regularization is needed: this is achieved directly through a penalty term, or indirectly by early stopping. Details are given in the next section.

The generic approach to minimizing $R(\theta)$ is by gradient descent, called *back-propagation* in this setting. Because of the compositional form of the model, the gradient can be easily derived using the chain rule for differentiation. This can be computed by a forward and backward sweep over the network, keeping track only of quantities local to each unit.

Here is back-propagation in detail for squared error loss. Let $z_{mi} = \sigma(\alpha_{0m} + \alpha_m^T x_i)$, from (11.5) and let $z_i = (z_{1i}, z_{2i}, \dots, z_{Mi})$. Then we have

$$\begin{aligned} R(\theta) &\equiv \sum_{i=1}^N R_i \\ &= \sum_{i=1}^N \sum_{k=1}^K (y_{ik} - f_k(x_i))^2, \end{aligned} \quad (11.11)$$

with derivatives

$$\begin{aligned} \frac{\partial R_i}{\partial \beta_{km}} &= -2(y_{ik} - f_k(x_i))g'_k(\beta_k^T z_i)z_{mi}, \\ \frac{\partial R_i}{\partial \alpha_{m\ell}} &= -\sum_{k=1}^K 2(y_{ik} - f_k(x_i))g'_k(\beta_k^T z_i)\beta_{km}\sigma'(\alpha_m^T x_i)x_{i\ell}. \end{aligned} \quad (11.12)$$

Given these derivatives, a gradient descent update at the $(r+1)$ st iteration has the form

$$\begin{aligned} \beta_{km}^{(r+1)} &= \beta_{km}^{(r)} - \gamma_r \sum_{i=1}^N \frac{\partial R_i}{\partial \beta_{km}^{(r)}}, \\ \alpha_{m\ell}^{(r+1)} &= \alpha_{m\ell}^{(r)} - \gamma_r \sum_{i=1}^N \frac{\partial R_i}{\partial \alpha_{m\ell}^{(r)}}, \end{aligned} \quad (11.13)$$

where γ_r is the *learning rate*, discussed below.

Now write (11.12) as

$$\begin{aligned} \frac{\partial R_i}{\partial \beta_{km}} &= \delta_{ki}z_{mi}, \\ \frac{\partial R_i}{\partial \alpha_{m\ell}} &= s_{mi}x_{i\ell}. \end{aligned} \quad (11.14)$$

The quantities δ_{ki} and s_{mi} are “errors” from the current model at the output and hidden layer units, respectively. From their definitions, these errors satisfy

$$s_{mi} = \sigma'(\alpha_m^T x_i) \sum_{k=1}^K \beta_{km} \delta_{ki}, \quad (11.15)$$

known as the *back-propagation equations*. Using this, the updates in (11.13) can be implemented with a two-pass algorithm. In the *forward pass*, the current weights are fixed and the predicted values $\hat{f}_k(x_i)$ are computed from formula (11.5). In the *backward pass*, the errors δ_{ki} are computed, and then back-propagated via (11.15) to give the errors s_{mi} . Both sets of errors are then used to compute the gradients for the updates in (11.13), via (11.14).

This two-pass procedure is what is known as back-propagation. It has also been called the *delta rule* (Widrow and Hoff, 1960). The computational components for cross-entropy have the same form as those for the sum of squares error function, and are derived in Exercise 11.3.

The advantages of back-propagation are its simple, local nature. In the back propagation algorithm, each hidden unit passes and receives information only to and from units that share a connection. Hence it can be implemented efficiently on a parallel architecture computer.

The updates in (11.13) are a kind of *batch learning*, with the parameter updates being a sum over all of the training cases. Learning can also be carried out online—processing each observation one at a time, updating the gradient after each training case, and cycling through the training cases many times. In this case, the sums in equations (11.13) are replaced by a single summand. A *training epoch* refers to one sweep through the entire training set. Online training allows the network to handle very large training sets, and also to update the weights as new observations come in.

The learning rate γ_r for batch learning is usually taken to be a constant, and can also be optimized by a line search that minimizes the error function at each update. With online learning γ_r should decrease to zero as the iteration $r \rightarrow \infty$. This learning is a form of *stochastic approximation* (Robbins and Munro, 1951); results in this field ensure convergence if $\gamma_r \rightarrow 0$, $\sum_r \gamma_r = \infty$, and $\sum_r \gamma_r^2 < \infty$ (satisfied, for example, by $\gamma_r = 1/r$).

Back-propagation can be very slow, and for that reason is usually not the method of choice. Second-order techniques such as Newton's method are not attractive here, because the second derivative matrix of R (the Hessian) can be very large. Better approaches to fitting include conjugate gradients and variable metric methods. These avoid explicit computation of the second derivative matrix while still providing faster convergence.

11.5 Some Issues in Training Neural Networks

There is quite an art in training neural networks. The model is generally overparametrized, and the optimization problem is nonconvex and unstable unless certain guidelines are followed. In this section we summarize some of the important issues.

11.5.1 Starting Values

Note that if the weights are near zero, then the operative part of the sigmoid (Figure 11.3) is roughly linear, and hence the neural network collapses into an approximately linear model (Exercise 11.2). Usually starting values for weights are chosen to be random values near zero. Hence the model starts out nearly linear, and becomes nonlinear as the weights increase. Individual

units localize to directions and introduce nonlinearities where needed. Use of exact zero weights leads to zero derivatives and perfect symmetry, and the algorithm never moves. Starting instead with large weights often leads to poor solutions.

11.5.2 Overfitting

Often neural networks have too many weights and will overfit the data at the global minimum of R . In early developments of neural networks, either by design or by accident, an early stopping rule was used to avoid overfitting. Here we train the model only for a while, and stop well before we approach the global minimum. Since the weights start at a highly regularized (linear) solution, this has the effect of shrinking the final model toward a linear model. A validation dataset is useful for determining when to stop, since we expect the validation error to start increasing.

A more explicit method for regularization is *weight decay*, which is analogous to ridge regression used for linear models (Section 3.4.1). We add a penalty to the error function $R(\theta) + \lambda J(\theta)$, where

$$J(\theta) = \sum_{k,m} \beta_{km}^2 + \sum_{m,\ell} \alpha_{m\ell}^2 \quad (11.16)$$

and $\lambda \geq 0$ is a tuning parameter. Larger values of λ will tend to shrink the weights toward zero: typically cross-validation is used to estimate λ . The effect of the penalty is to simply add terms $2\beta_{km}$ and $2\alpha_{m\ell}$ to the respective gradient expressions (11.13). Other forms for the penalty have been proposed, for example,

$$J(\theta) = \sum_{k,m} \frac{\beta_{km}^2}{1 + \beta_{km}^2} + \sum_{m,\ell} \frac{\alpha_{m\ell}^2}{1 + \alpha_{m\ell}^2}, \quad (11.17)$$

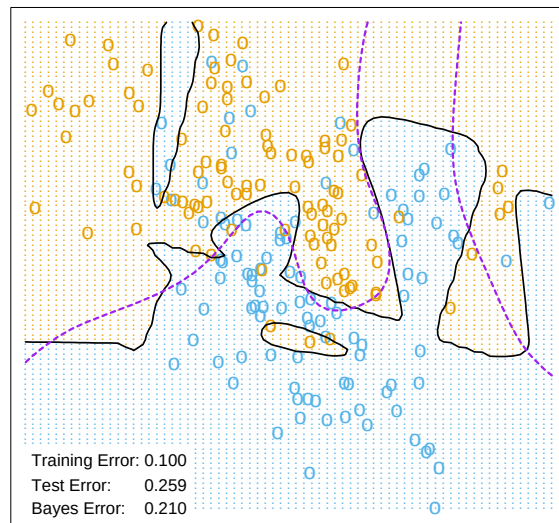
known as the *weight elimination* penalty. This has the effect of shrinking smaller weights more than (11.16) does.

Figure 11.4 shows the result of training a neural network with ten hidden units, without weight decay (upper panel) and with weight decay (lower panel), to the mixture example of Chapter 2. Weight decay has clearly improved the prediction. Figure 11.5 shows heat maps of the estimated weights from the training (grayscale versions of these are called *Hinton diagrams*.) We see that weight decay has dampened the weights in both layers: the resulting weights are spread fairly evenly over the ten hidden units.

11.5.3 Scaling of the Inputs

Since the scaling of the inputs determines the effective scaling of the weights in the bottom layer, it can have a large effect on the quality of the final

Neural Network - 10 Units, No Weight Decay



Neural Network - 10 Units, Weight Decay=0.02

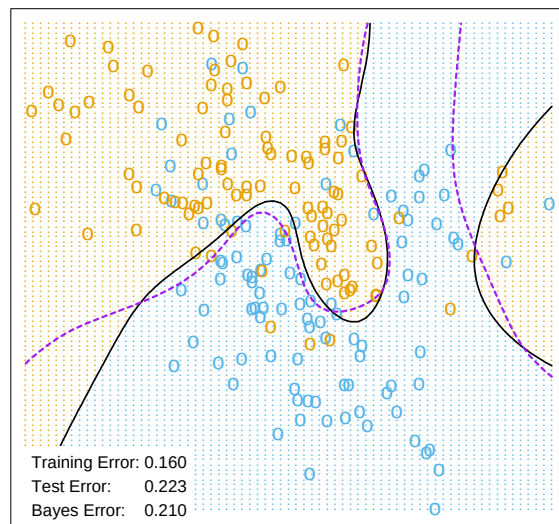


FIGURE 11.4. A neural network on the mixture example of Chapter 2. The upper panel uses no weight decay, and overfits the training data. The lower panel uses weight decay, and achieves close to the Bayes error rate (broken purple boundary). Both use the softmax activation function and cross-entropy error.

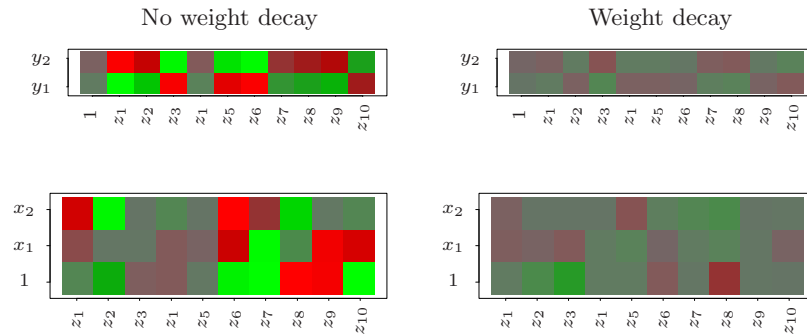


FIGURE 11.5. Heat maps of the estimated weights from the training of neural networks from Figure 11.4. The display ranges from bright green (negative) to bright red (positive).

solution. At the outset it is best to standardize all inputs to have mean zero and standard deviation one. This ensures all inputs are treated equally in the regularization process, and allows one to choose a meaningful range for the random starting weights. With standardized inputs, it is typical to take random uniform weights over the range $[-0.7, +0.7]$.

11.5.4 Number of Hidden Units and Layers

Generally speaking it is better to have too many hidden units than too few. With too few hidden units, the model might not have enough flexibility to capture the nonlinearities in the data; with too many hidden units, the extra weights can be shrunk toward zero if appropriate regularization is used. Typically the number of hidden units is somewhere in the range of 5 to 100, with the number increasing with the number of inputs and number of training cases. It is most common to put down a reasonably large number of units and train them with regularization. Some researchers use cross-validation to estimate the optimal number, but this seems unnecessary if cross-validation is used to estimate the regularization parameter. Choice of the number of hidden layers is guided by background knowledge and experimentation. Each layer extracts features of the input for regression or classification. Use of multiple hidden layers allows construction of hierarchical features at different levels of resolution. An example of the effective use of multiple layers is given in Section 11.6.

11.5.5 Multiple Minima

The error function $R(\theta)$ is nonconvex, possessing many local minima. As a result, the final solution obtained is quite dependent on the choice of start-

ing weights. One must at least try a number of random starting configurations, and choose the solution giving lowest (penalized) error. Probably a better approach is to use the average predictions over the collection of networks as the final prediction (Ripley, 1996). This is preferable to averaging the weights, since the nonlinearity of the model implies that this averaged solution could be quite poor. Another approach is via *bagging*, which averages the predictions of networks training from randomly perturbed versions of the training data. This is described in Section 8.7.

11.6 Example: Simulated Data

We generated data from two additive error models $Y = f(X) + \varepsilon$:

$$\begin{aligned} \text{Sum of sigmoids: } Y &= \sigma(a_1^T X) + \sigma(a_2^T X) + \varepsilon_1; \\ \text{Radial: } Y &= \prod_{m=1}^{10} \phi(X_m) + \varepsilon_2. \end{aligned}$$

Here $X^T = (X_1, X_2, \dots, X_p)$, each X_j being a standard Gaussian variate, with $p = 2$ in the first model, and $p = 10$ in the second.

For the sigmoid model, $a_1 = (3, 3)$, $a_2 = (3, -3)$; for the radial model, $\phi(t) = (1/2\pi)^{1/2} \exp(-t^2/2)$. Both ε_1 and ε_2 are Gaussian errors, with variance chosen so that the signal-to-noise ratio

$$\frac{\text{Var}(E(Y|X))}{\text{Var}(Y - E(Y|X))} = \frac{\text{Var}(f(X))}{\text{Var}(\varepsilon)} \quad (11.18)$$

is 4 in both models. We took a training sample of size 100 and a test sample of size 10,000. We fit neural networks with weight decay and various numbers of hidden units, and recorded the average test error $E_{\text{Test}}(Y - \hat{f}(X))^2$ for each of 10 random starting weights. Only one training set was generated, but the results are typical for an “average” training set. The test errors are shown in Figure 11.6. Note that the zero hidden unit model refers to linear least squares regression. The neural network is perfectly suited to the sum of sigmoids model, and the two-unit model does perform the best, achieving an error close to the Bayes rate. (Recall that the Bayes rate for regression with squared error is the error variance; in the figures, we report test error relative to the Bayes error). Notice, however, that with more hidden units, overfitting quickly creeps in, and with some starting weights the model does worse than the linear model (zero hidden unit) model. Even with two hidden units, two of the ten starting weight configurations produced results no better than the linear model, confirming the importance of multiple starting values.

A radial function is in a sense the most difficult for the neural net, as it is spherically symmetric and with no preferred directions. We see in the right

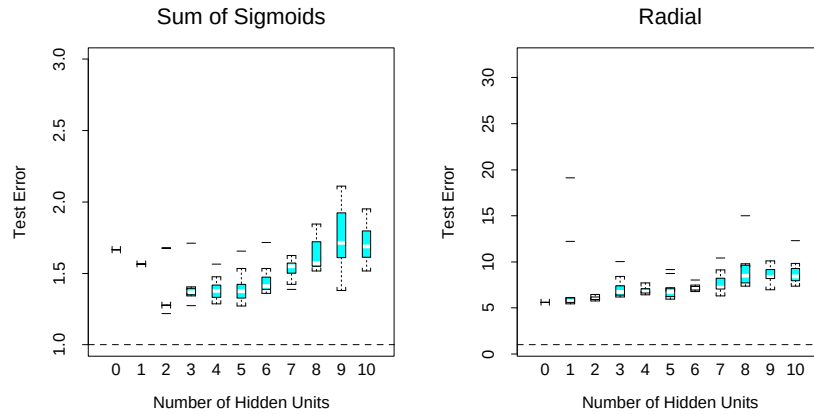


FIGURE 11.6. Boxplots of test error, for simulated data example, relative to the Bayes error (broken horizontal line). True function is a sum of two sigmoids on the left, and a radial function is on the right. The test error is displayed for 10 different starting weights, for a single hidden layer neural network with the number of units as indicated.

panel of Figure 11.6 that it does poorly in this case, with the test error staying well above the Bayes error (note the different vertical scale from the left panel). In fact, since a constant fit (such as the sample average) achieves a relative error of 5 (when the SNR is 4), we see that the neural networks perform increasingly worse than the mean.

In this example we used a fixed weight decay parameter of 0.0005, representing a mild amount of regularization. The results in the left panel of Figure 11.6 suggest that more regularization is needed with greater numbers of hidden units.

In Figure 11.7 we repeated the experiment for the sum of sigmoids model, with no weight decay in the left panel, and stronger weight decay ($\lambda = 0.1$) in the right panel. With no weight decay, overfitting becomes even more severe for larger numbers of hidden units. The weight decay value $\lambda = 0.1$ produces good results for all numbers of hidden units, and there does not appear to be overfitting as the number of units increase. Finally, Figure 11.8 shows the test error for a ten hidden unit network, varying the weight decay parameter over a wide range. The value 0.1 is approximately optimal.

In summary, there are two free parameters to select: the weight decay λ and number of hidden units M . As a learning strategy, one could fix either parameter at the value corresponding to the least constrained model, to ensure that the model is rich enough, and use cross-validation to choose the other parameter. Here the least constrained values are zero weight decay and ten hidden units. Comparing the left panel of Figure 11.7 to Figure 11.8, we see that the test error is less sensitive to the value of the weight

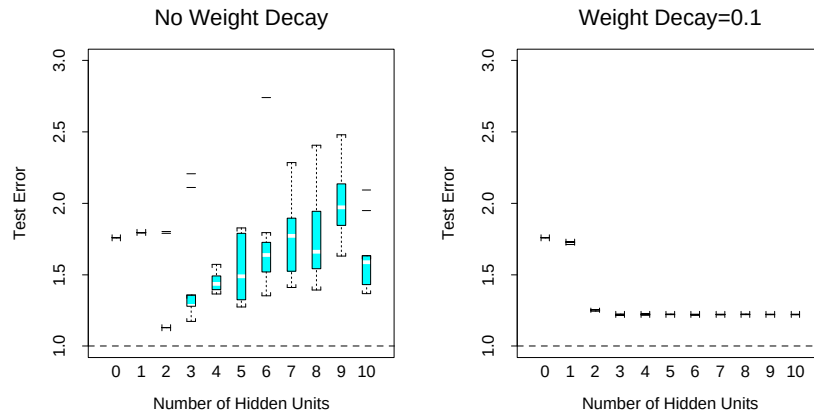


FIGURE 11.7. Boxplots of test error, for simulated data example, relative to the Bayes error. True function is a sum of two sigmoids. The test error is displayed for ten different starting weights, for a single hidden layer neural network with the number units as indicated. The two panels represent no weight decay (left) and strong weight decay $\lambda = 0.1$ (right).

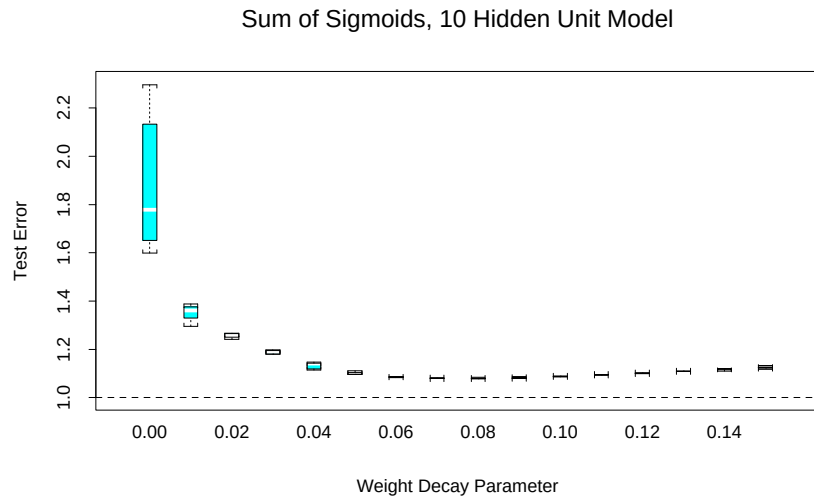


FIGURE 11.8. Boxplots of test error, for simulated data example. True function is a sum of two sigmoids. The test error is displayed for ten different starting weights, for a single hidden layer neural network with ten hidden units and weight decay parameter value as indicated.

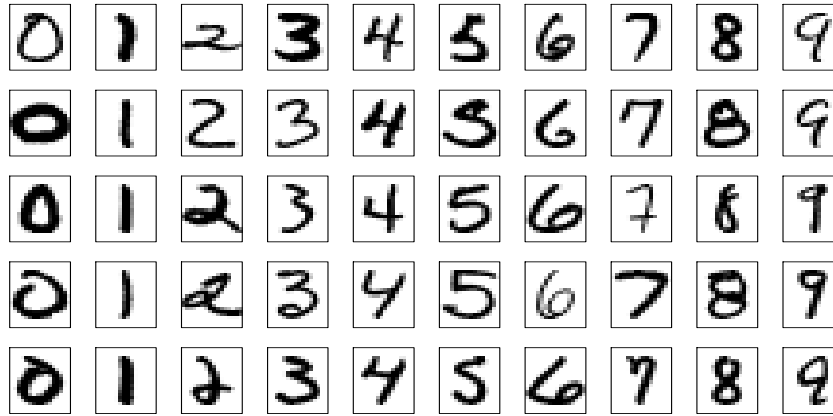


FIGURE 11.9. Examples of training cases from ZIP code data. Each image is a 16×16 8-bit grayscale representation of a handwritten digit.

decay parameter, and hence cross-validation of this parameter would be preferred.

11.7 Example: ZIP Code Data

This example is a character recognition task: classification of handwritten numerals. This problem captured the attention of the machine learning and neural network community for many years, and has remained a benchmark problem in the field. Figure 11.9 shows some examples of normalized handwritten digits, automatically scanned from envelopes by the U.S. Postal Service. The original scanned digits are binary and of different sizes and orientations; the images shown here have been deslanted and size normalized, resulting in 16×16 grayscale images (Le Cun et al., 1990). These 256 pixel values are used as inputs to the neural network classifier.

A *black box* neural network is not ideally suited to this pattern recognition task, partly because the pixel representation of the images lack certain invariances (such as small rotations of the image). Consequently early attempts with neural networks yielded misclassification rates around 4.5% on various examples of the problem. In this section we show some of the pioneering efforts to handcraft the neural network to overcome some these deficiencies (Le Cun, 1989), which ultimately led to the state of the art in neural network performance (Le Cun et al., 1998)¹.

Although current digit datasets have tens of thousands of training and test examples, the sample size here is deliberately modest in order to em-

¹The figures and tables in this example were recreated from Le Cun (1989).

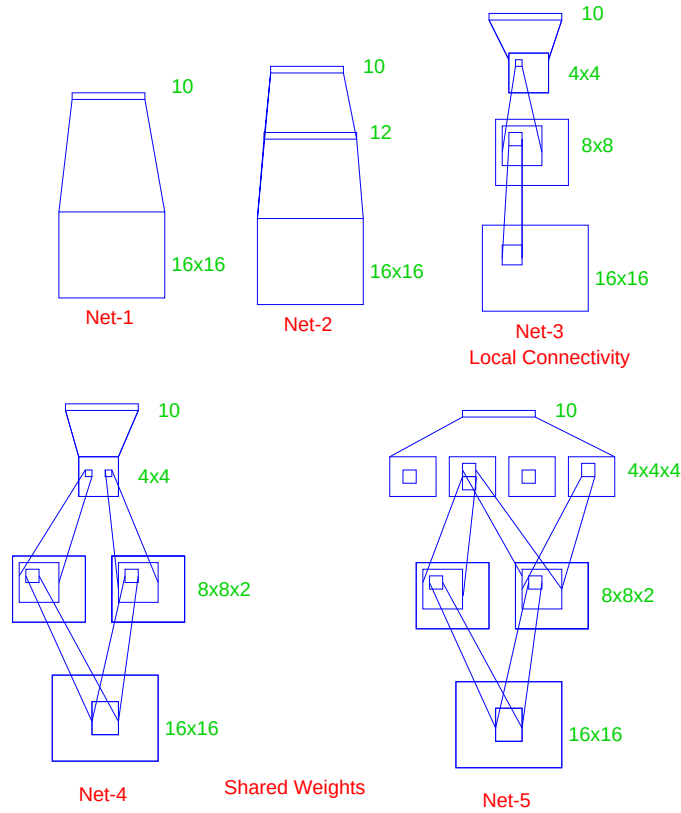


FIGURE 11.10. Architecture of the five networks used in the ZIP code example.

phasize the effects. The examples were obtained by scanning some actual hand-drawn digits, and then generating additional images by random horizontal shifts. Details may be found in Le Cun (1989). There are 320 digits in the training set, and 160 in the test set.

Five different networks were fit to the data:

Net-1: No hidden layer, equivalent to multinomial logistic regression.

Net-2: One hidden layer, 12 hidden units fully connected.

Net-3: Two hidden layers locally connected.

Net-4: Two hidden layers, locally connected with weight sharing.

Net-5: Two hidden layers, locally connected, two levels of weight sharing.

These are depicted in Figure 11.10. Net-1 for example has 256 inputs, one each for the 16×16 input pixels, and ten output units for each of the digits 0–9. The predicted value $\hat{f}_k(x)$ represents the estimated probability that an image x has digit class k , for $k = 0, 1, 2, \dots, 9$.